Connections between HJ Reachability Analysis and Control Barrier Functions

Merging two ways of building a safety value function

Sylvia Herbert (UC San Diego) · Autonomy Talks, ETH Zürich (online), 11 January 2022
Video: https://www.youtube.com/watch?v=-PzULKLB0D4

THE TALK IN 5 POINTS

- **Shared idea:** both methods build one scalar *safety value function* of the state. Positive means safe, zero is the edge of the safe set, and the gradient points toward more safety.
- **HJ reachability** builds it by brute-force dynamic programming on a grid. It is general (control and disturbance bounds, reach-avoid, moving obstacles) and finds the *largest* safe set. But grid size explodes with state dimension (about 5–6 states max), and online use is either too cautious or jerky.
- **CBFs** only require $\dot{B} \geq -\gamma B$: safety may drop when far from the boundary but not near it. This gives a smooth, cheap safety filter (the CBF-QP). But there is no general recipe for finding a valid CBF, and hand-made ones cover less of the safe set.
- **CBVF** (new): add the CBF decay rate γ into the reachability equation. The result B_γ is built by dynamic programming (constructive, handles bounds, largest safe set) and plugs into a robust CBVF-QP filter.
- **Next:** finite-time safety, reachability + control Lyapunov functions, safe learning that mixes CBF approximations with reachability tools, and multi-agent CBFs. Q&A covers recovering after leaving the safe set and “soft” safety.

CONTENTS

1. Motivation · p. 2
2. HJ reachability: building the value function · p. 7
3. HJ reachability: using it online · p. 18
4. Control barrier functions · p. 26
5. Merging them: the CBVF · p. 34
6. Future work and questions · p. 41

TERMS

Value function: a number for each state; here ≥ 0 means safe.

HJ reachability: finds safe sets by solving a PDE (partial differential equation) backward in time on a grid.

BRS / BRT: backward reachable set / tube: states that hit the target at exactly time T / at any time up to T.

HJ-VI: HJ variational inequality, the reachability PDE with an extra min{l − V, …} term.

CBF: control barrier function B with $\dot{B} + \gamma B \geq 0$.

QP: quadratic program, a small, fast optimization solved at each control step.

CBVF: control barrier-value function, reachability with the CBF decay rate γ built in.

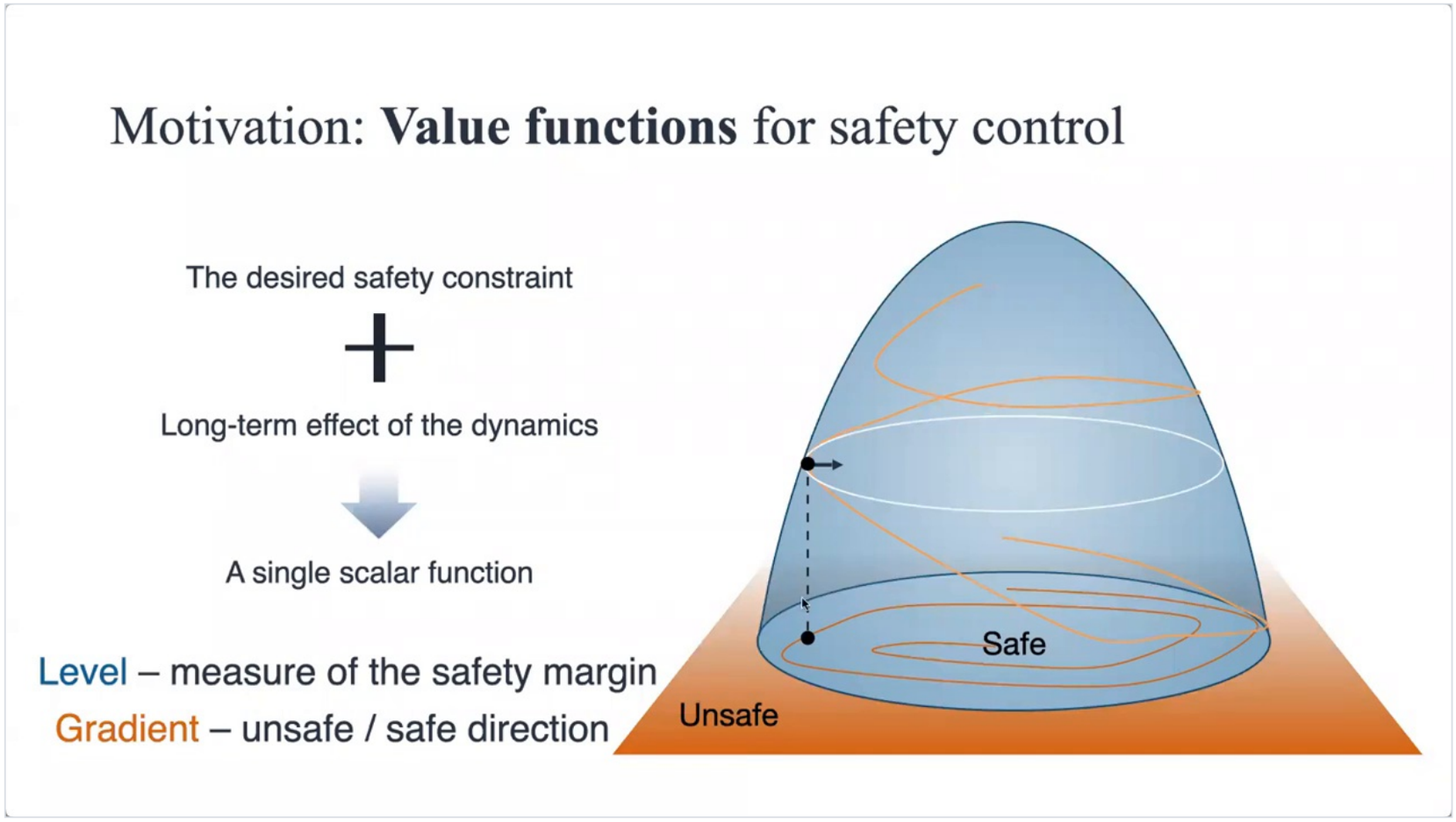


1. MOTIVATION

Title and introduction

▶ 0:00 · intro from 0:00

- The Autonomy Talks host introduces Sylvia Herbert: assistant professor of mechanical and aerospace engineering at UC San Diego, PhD with Claire Tomlin at UC Berkeley.
- Origin of the work: teaching a safety course at UCSD forced her to compare the two methods carefully, and it became a research project.
- Joint work with PhD students Jason Choi and Donggun Lee, Koushil Sreenath (the CBF expert) and Claire Tomlin (the reachability expert). Many slides come from Jason Choi's CDC talk.



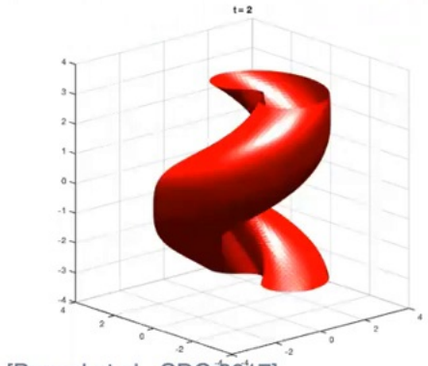
1. MOTIVATION

Motivation: value functions for safety

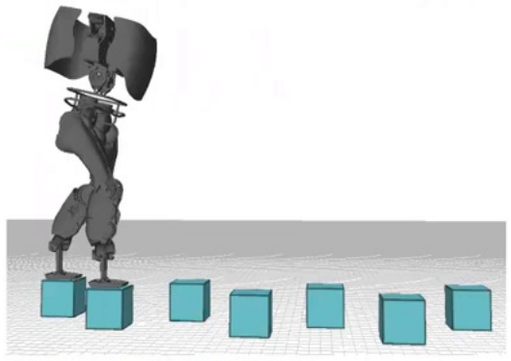
▶ 2:21

- Goal of both fields: turn a safety constraint, plus the long-term effect of the dynamics, into one scalar function of the state.
- **Level** = safety margin: positive inside the safe set, zero at its edge, negative when unsafe.
- **Gradient** = which way is safer. Online, you can read how safe you are now and which direction recovers safety.

Two popular value-function based approaches:
Hamilton-Jacobi Reachability and Control Barrier Functions



[Bansal et al., CDC 2017]



[Nguyen et al., CDC 2016]

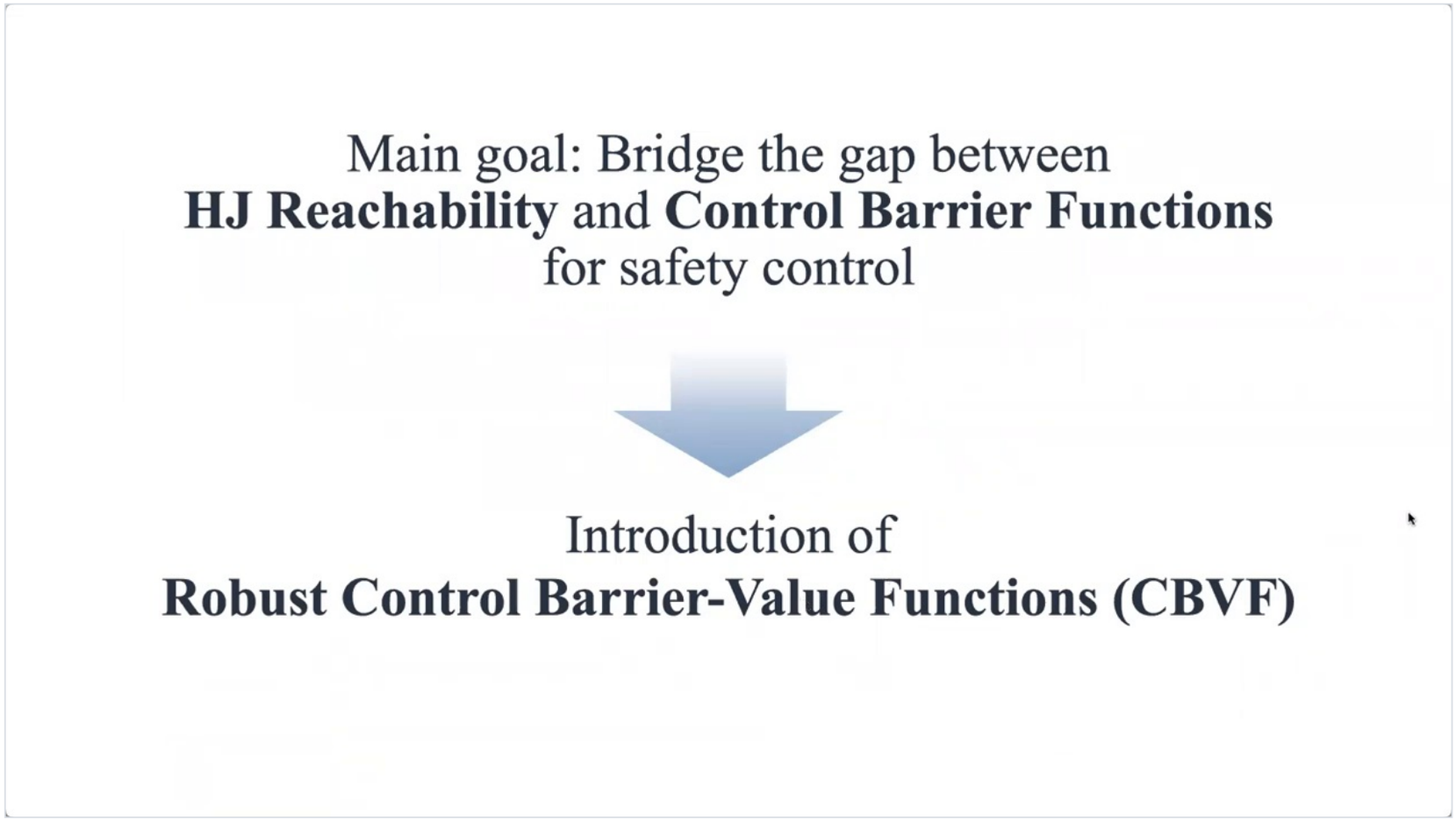
What is the relationship between these two methods?
Can we migrate theory and tools across subfields?

1. MOTIVATION

Two popular value-function approaches

▶ 3:34

- Hamilton-Jacobi (HJ) reachability (left, Bansal et al., CDC 2017) and control barrier functions (right, a walking robot on stepping stones, Nguyen et al., CDC 2016) both build and use such a function.
- The two grew up separately with different tools. Questions: how are they related, and can theory and tools move between them?

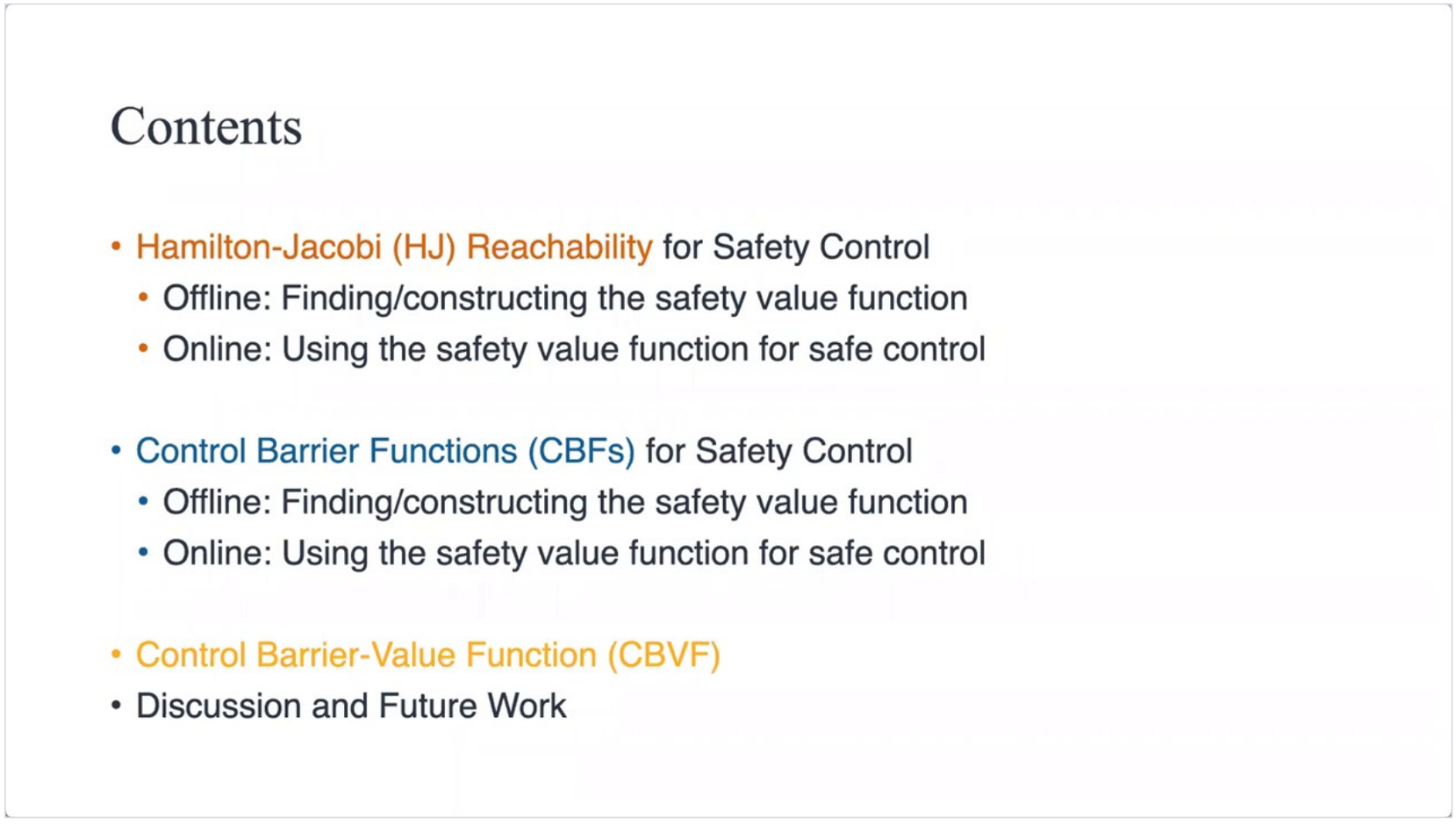


1. MOTIVATION

Main goal

▶ 4:10

- Bridge the gap between HJ reachability and CBFs for safety control.
- Outcome: a mix of the two, the **robust control barrier-value function (CBVF)**.



1. MOTIVATION

Contents

▶ 4:27 · revisited at 50:19

- Part 1: HJ reachability. Offline: build the safety value function. Online: use it for safe control.
- Part 2: CBFs, with the same offline/online split.
- Part 3: the CBVF. Part 4: discussion and future work.
- Shown again at 50:19 to start the future-work part.

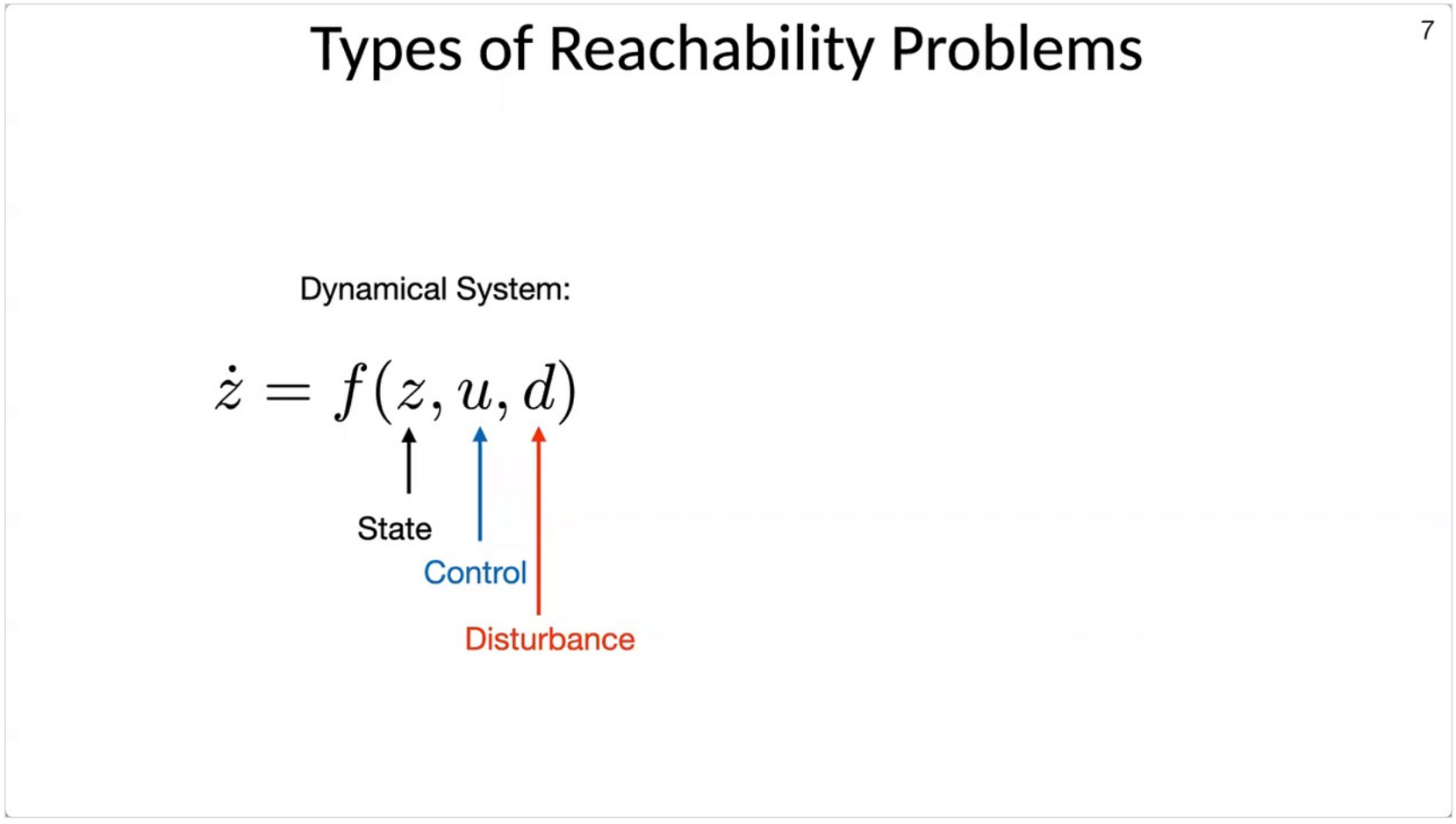


2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

Part 1 starts: HJ reachability, offline

▶ 5:09

- First topic: how reachability builds (offline) the safety value function.

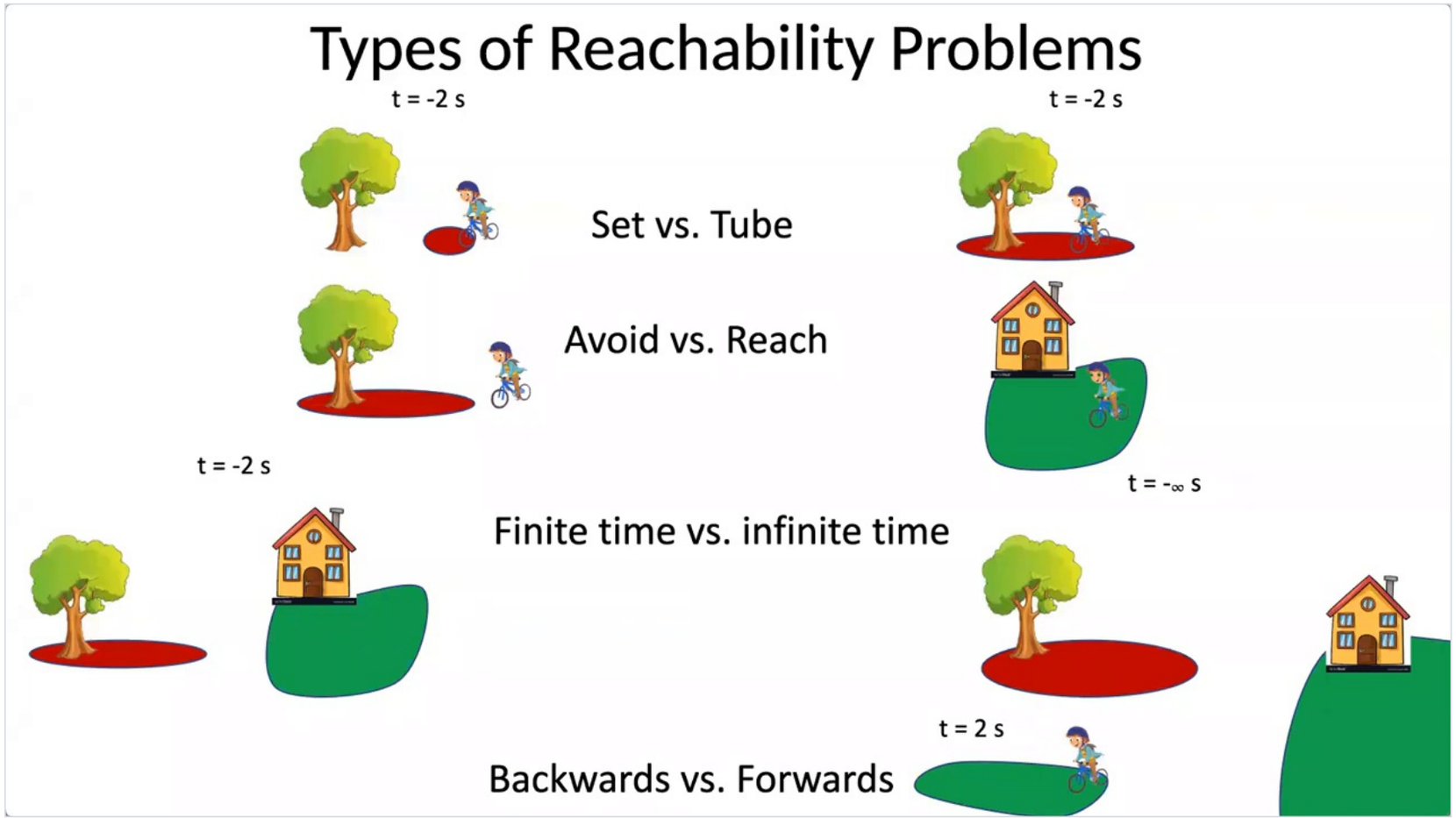


2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

System model

▶ 5:18

- Reachability assumes a general system ż = f(z, u, d): state z, control u, disturbance d.
- Usual setup: control and disturbance are bounded, and the disturbance is assumed to act in the worst way. Stochastic versions exist (e.g., CVaR, a risk measure) but are not covered.
- Because the method is brute-force dynamic programming, it handles many problem types (next slide).

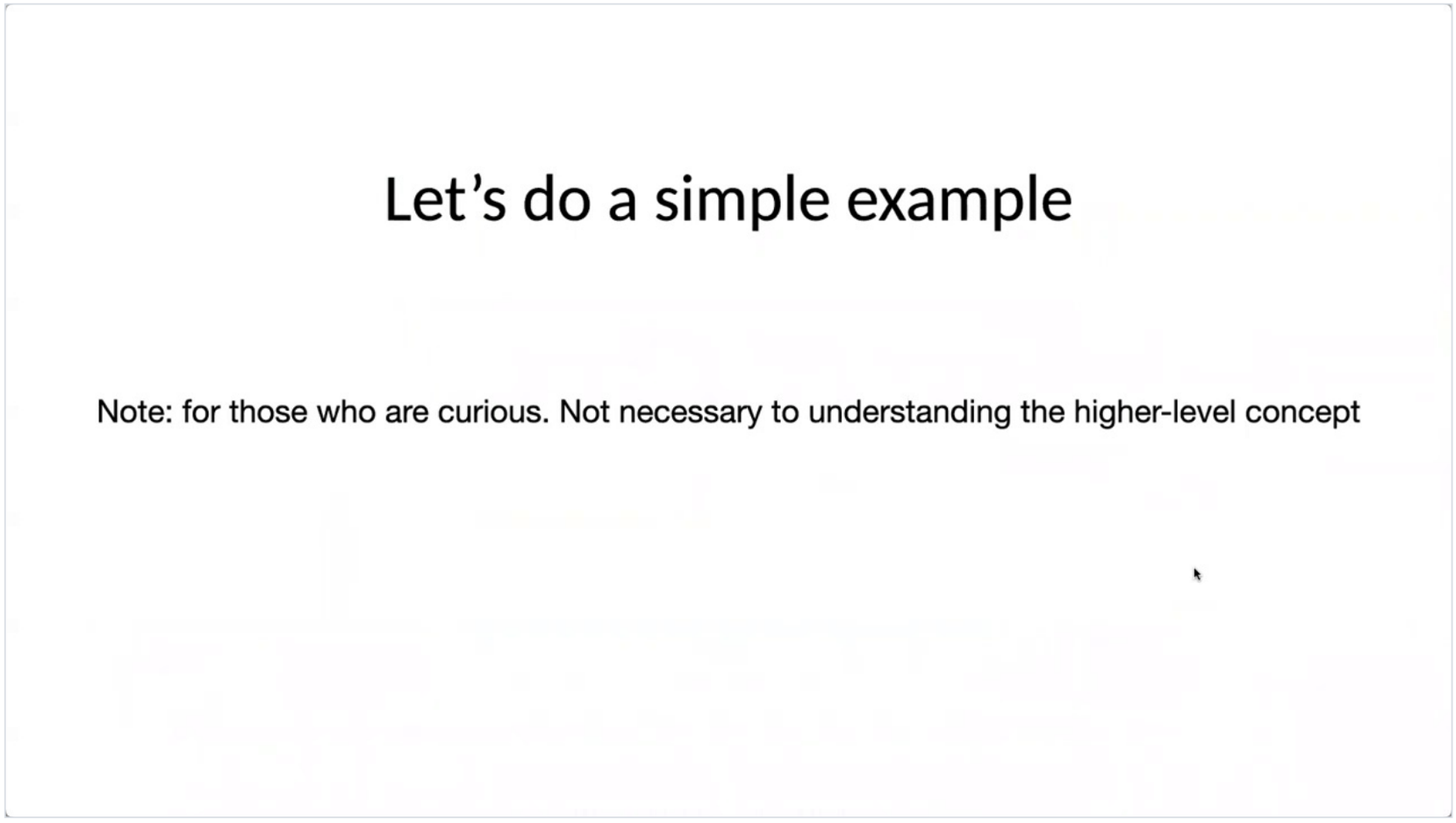


2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

Types of reachability problems

▶ 6:09

- **Set vs. tube**: states that hit the tree at *exactly* 2 s, vs. *at any time within* 2 s (the tube includes the tree itself). Safety usually needs the tube.
- **Avoid vs. reach**: control tries to stay away while the disturbance pushes in, or control tries to reach a goal (the house).
- **Finite vs. infinite time**: avoid sets usually converge ("I can avoid a tree in Switzerland from California"); reach sets usually keep growing.
- **Backward vs. forward**: work back from the target or forward from the start. These combine (reach-avoid, moving goals and obstacles).
- Rest of the talk: infinite-time avoid, the case CBFs handle.

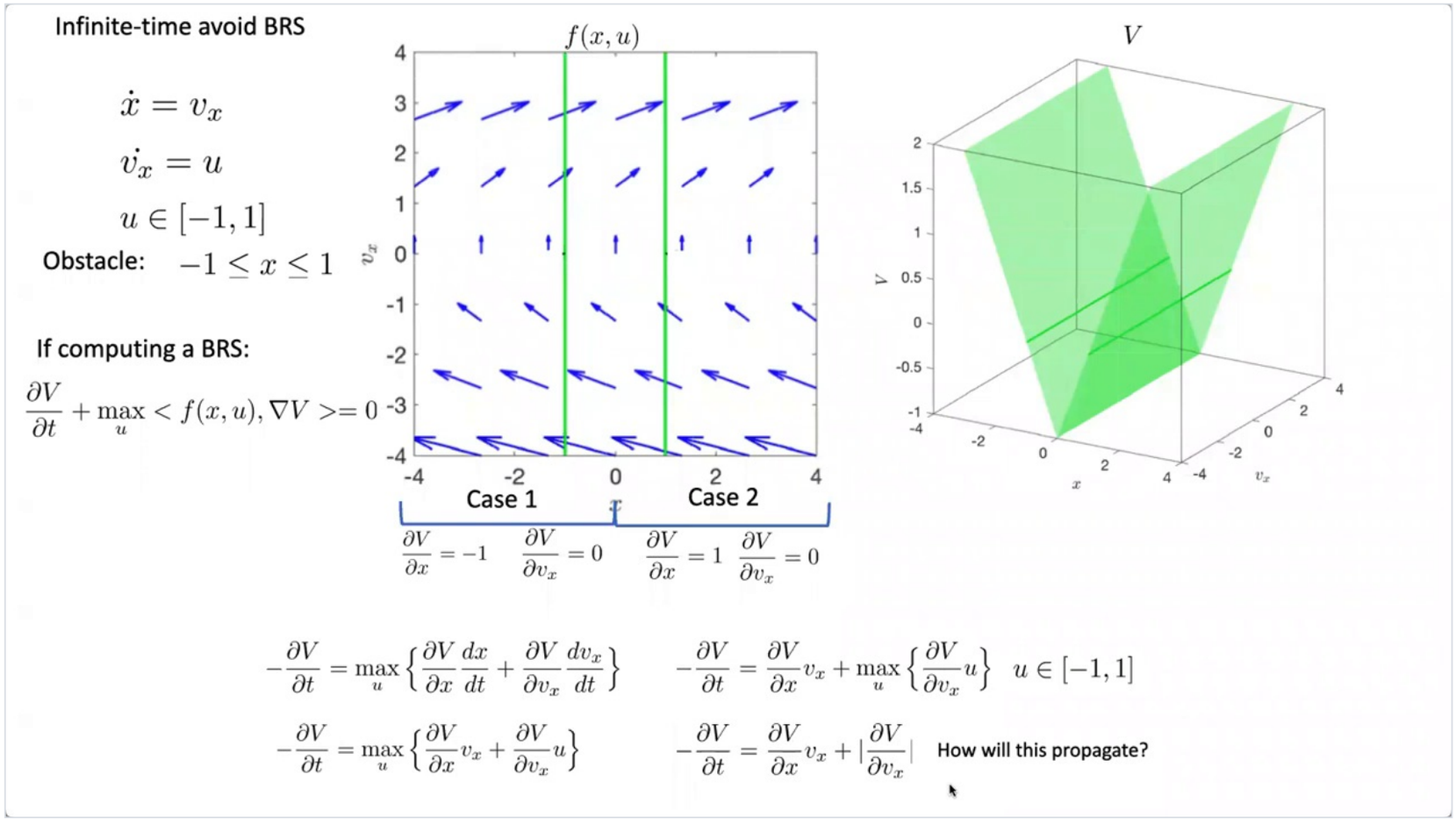


2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

A simple example

▶ 10:03

- Optional toy example of how reachability is actually computed in code. Not needed for the main ideas.



2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

Toy example: the value function update

▶ 10:20

- Dynamics: $dx/dt = v_x$, $dv_x/dt = u$, with $u \in [-1, 1]$. Obstacle: $-1 \leq x \leq 1$, at every velocity.
- Start from $l(x)$, a signed distance to the obstacle (negative inside, positive outside). Use it as the value at the final time and step backward.
- Update (HJ PDE): $\partial V/\partial t + \max_u f(x,u)\cdot\nabla V = 0$. The control tries to climb the value gradient while obeying the dynamics.
- With $|u| \leq 1$ the max picks $u = \text{sign}(\partial V/\partial v_x)$, so: $-\partial V/\partial t = (\partial V/\partial x)\cdot v_x + |\partial V/\partial v_x|$.

Infinite-time avoid BRS

$$\dot{x} = v_x$$

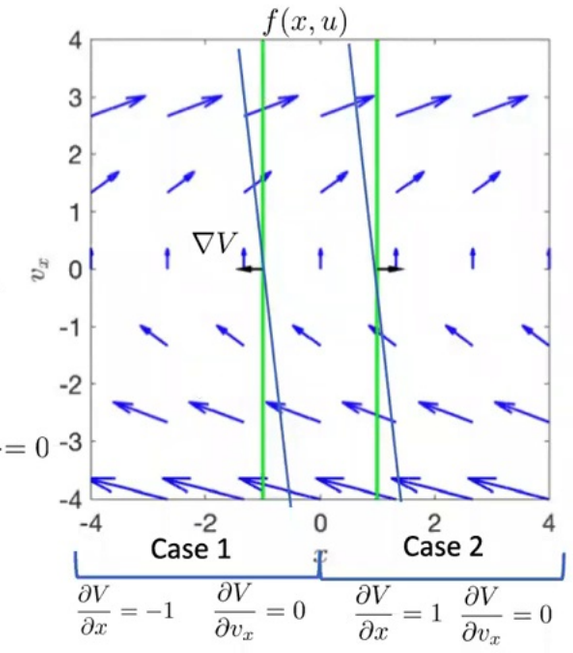
$$\dot{v_x} = u$$

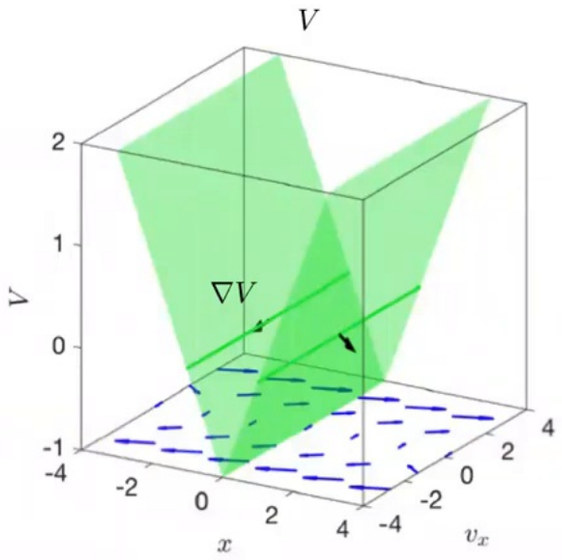
$$u \in [-1, 1]$$

Obstacle: $-1 \leq x \leq 1$

If computing a BRS:

$$\frac{\partial V}{\partial t} + \max_u < f(x,u), \nabla V >= 0$$





Case 1: $\frac{\partial V}{\partial x} = -1$ $\frac{\partial V}{\partial v_x} = 0$

Case 2: $\frac{\partial V}{\partial x} = 1$ $\frac{\partial V}{\partial v_x} = 0$

$$-\frac{\partial V}{\partial t} = \frac{\partial V}{\partial x} v_x + |\frac{\partial V}{\partial v_x}|$$

$$-\frac{\partial V}{\partial t} = (-1)v_x + 0$$

$$-\frac{\partial V}{\partial t} = (1)v_x + 0$$

How will this propagate?

$$-\frac{\partial V}{\partial t} = -v_x$$

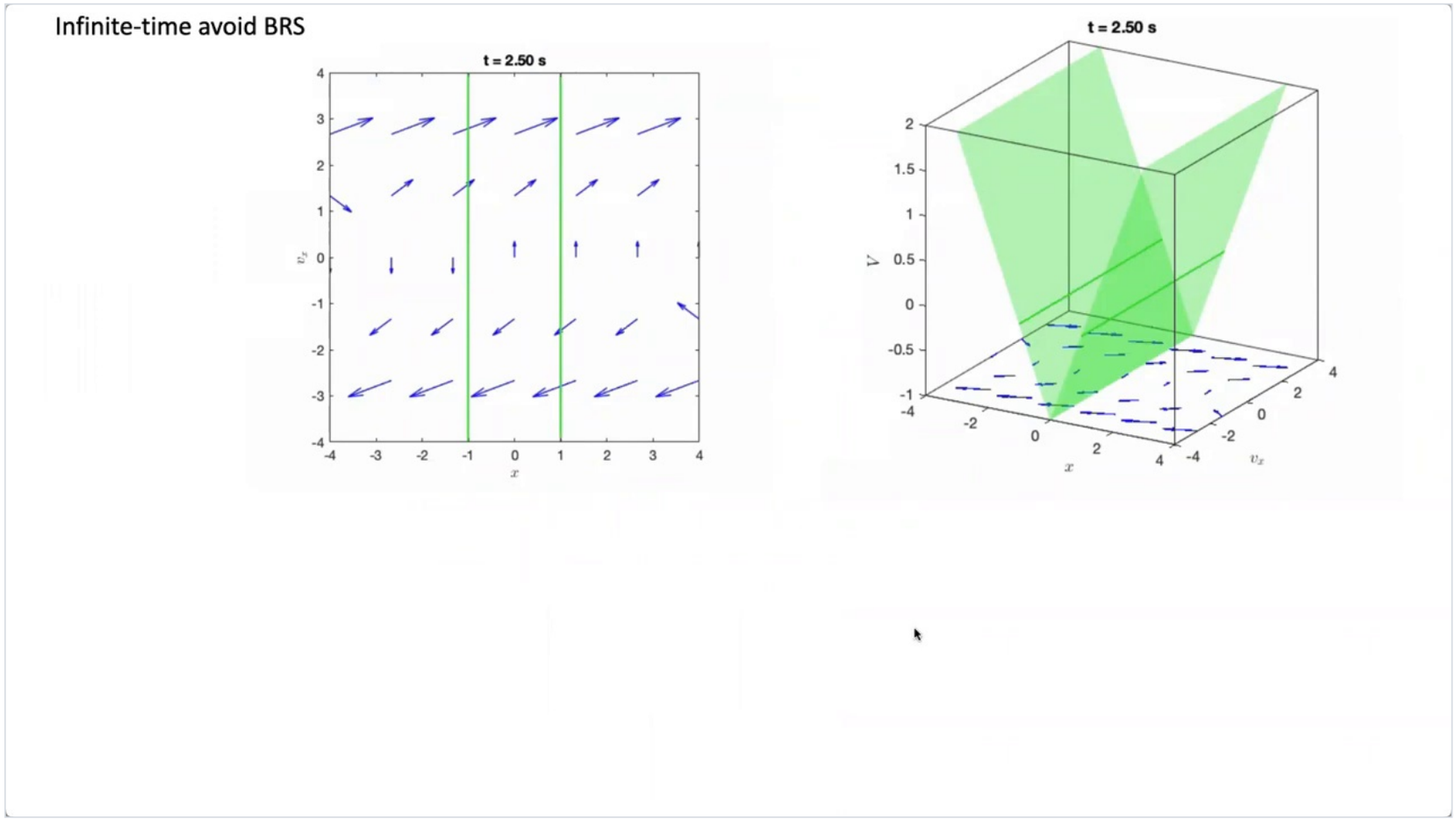
$$-\frac{\partial V}{\partial t} = v_x$$

2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

Toy example: one step backward

▶ 14:19

- Left of the obstacle the slope is ∂V/∂x = −1, so −∂V/∂t = −v_x. On the right the slope is +1, so −∂V/∂t = +v_x.
- Example: a boundary point at v_x = 3 shifts by its velocity, so the zero boundary tilts (rotates).
- Meaning after 1 s: a state right of the obstacle moving left fast will hit it even with full braking; a state inside it moving fast enough can get out.



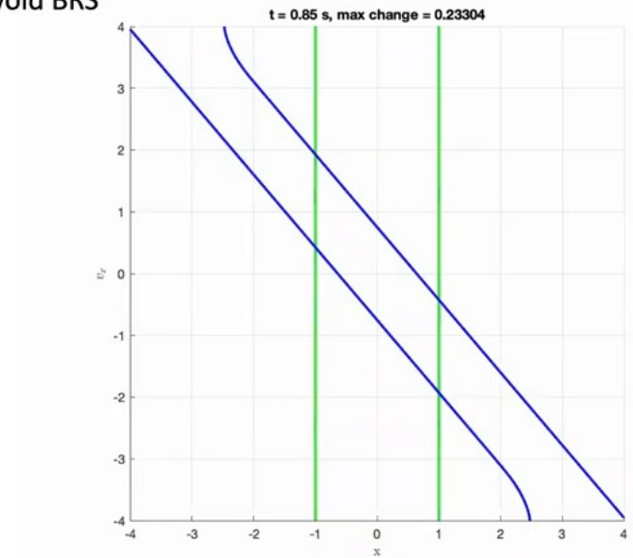
2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

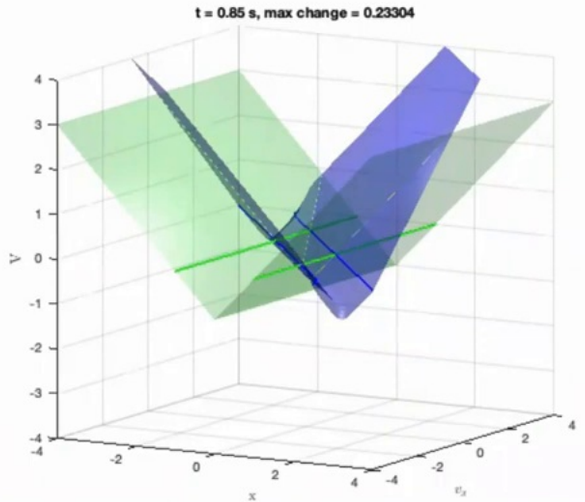
Toy example: running it longer

▶ 16:02

- Propagating further (animation, 0 → 2.5 s), the boundary keeps rotating, as expected.
- A second effect also appears: the whole value function lifts up (explained next).

Infinite-time avoid BRS





BRS: $$-\frac{\partial V}{\partial t} = \frac{\partial V}{\partial x} v_x + \left|\frac{\partial V}{\partial v_x}\right|$$

Gets weaker Gets stronger

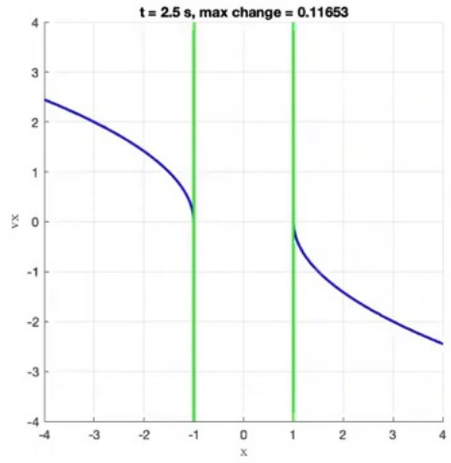
2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

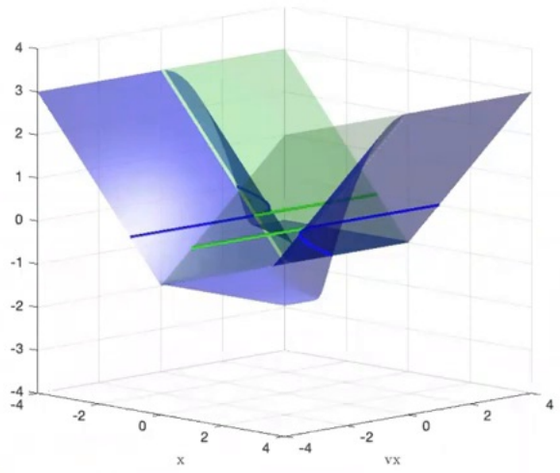
Why the reachable set disappears

▶ 16:19

- As the boundary rotates, ∂V/∂x “gets weaker” and $|\partial V/\partial v_x|$ “gets stronger”, so the value lifts everywhere.
- A backward reachable **set** (BRS) asks whether you are inside the obstacle at *exactly* time T. Given enough time you can always brake early or drive through it, so the set shrinks to nothing.
- For safety we care whether you *ever* touch the obstacle. That needs a tube.

Infinite-time avoid BRT





Now let's do a BRT:

BRS: $-\frac{\partial V}{\partial t} = \frac{\partial V}{\partial x} v_x + |\frac{\partial V}{\partial v_x}|$

New addition

$$\min\left\{\frac{\partial V}{\partial t} + \max_u < f(x,u), \nabla V >, l(x) - V(x)\right\} = 0$$

When $l(x) - V(x) < \frac{\partial V}{\partial t} + \max_u < f(x,u), \nabla V >$

We set $l(x) - V(x) = 0$

$$V(x) = l(x)$$

In other words, when the Hamiltonian is trying to push the value function into the obstacle (above l(x)), we force the function to be on the boundary of the obstacle (V(x) = l(x))

2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

Backward reachable tube (BRT)

▶ 17:54

- Same PDE plus one new term: min{ ∂V/∂t + max$_u$ f·∇V, l(x) − V(x) } = 0.
- Effect: whenever the update would push V above l(x), V is clamped to l(x). A state that was unsafe can never become safe again.
- Run to convergence: the result is every state that will *eventually* hit the obstacle, e.g., left of it and moving right too fast to brake.
- Works the same way in any state dimension; 2D (position, velocity) is just for pictures.

Tl;dr: We encode the safety constraint into the construction of the value function and apply dynamic programming

HJ-VI

$$0 \quad \leq \left\{ l(x) - V(x,t),\ D_t V(x,t) + \max_{u \in U} \min_{d \in D} D_x V(x,t) \cdot f(x,u,d) \right\}$$

$$\pi^*_V(x,\ t)$$

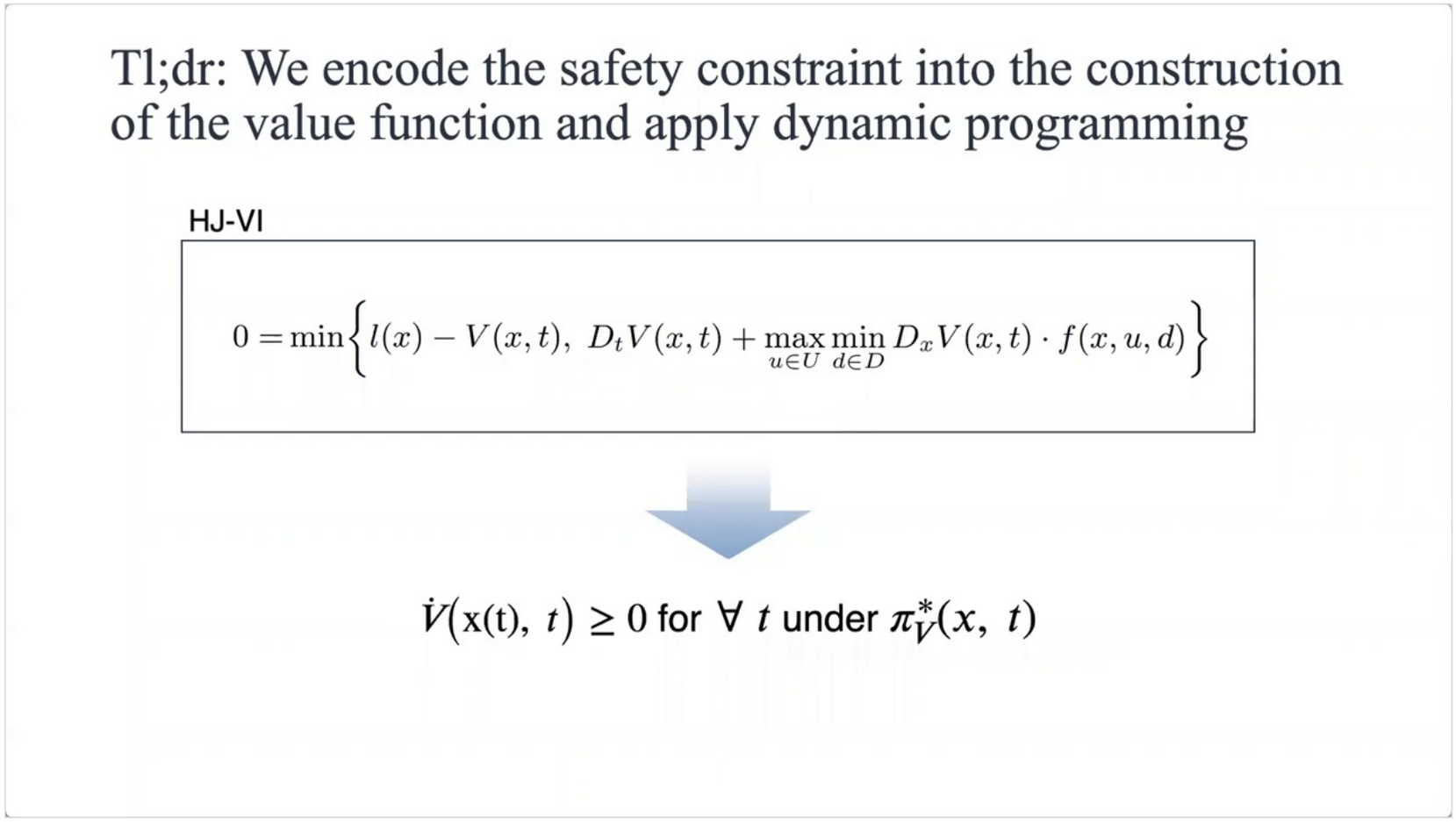
$$\dot{V}(\mathrm{x}(t),t) = D_t V(\mathrm{x}(t),t) + D_x V(\mathrm{x}(t),t) \cdot f(\mathrm{x}(t), \pi^*_V(\mathrm{x}(t),t), d) \geq 0,$$

2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

Tl;dr: the HJ variational inequality

▶ 20:22

- Reachability writes the safety constraint into the value function, then solves for it with dynamic programming.
- Key term of the HJ-VI: $D_t V + \max_u \min_d D_x V \cdot f(x,u,d) \geq 0$. By the chain rule this is just dV/dt along a trajectory under the optimal policy π^*_V.
- So the safety level can never drop: you stay at the same level or get safer.



2. HJ REACHABILITY: BUILDING THE VALUE FUNCTION

Tl;dr: full equation

▶ 21:39

- Full HJ-VI: 0 = min{ l(x) − V(x,t), D_tV + $\max_u$ $\min_d$ D_xV·f(x,u,d) }.
- Guarantee: dV/dt ≥ 0 for all t under the optimal policy π^*_V(x,t), found by brute-force dynamic programming.

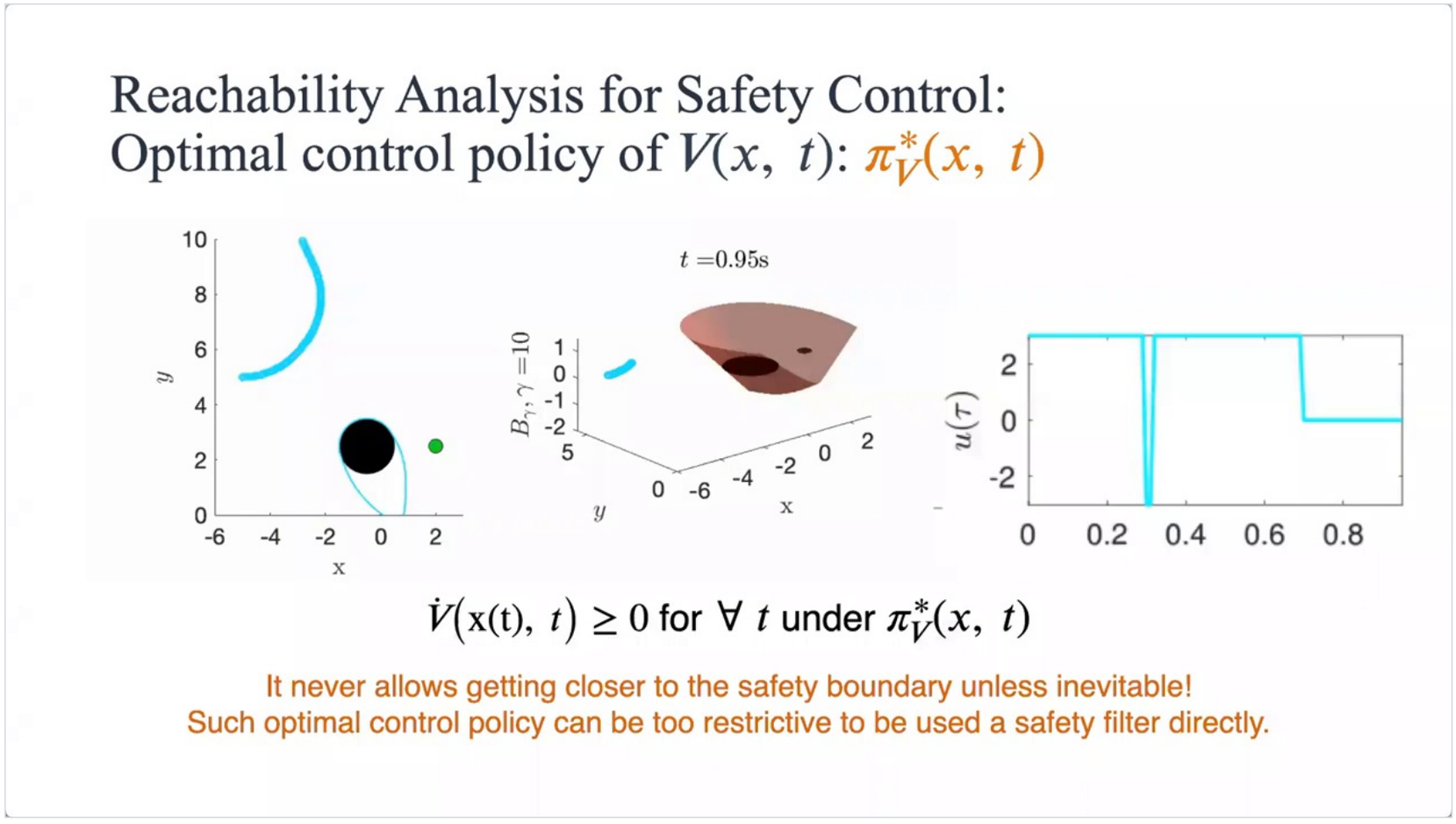


3. HJ REACHABILITY: USING IT ONLINE

Part 1b: HJ reachability, online

▶ 21:53

- Next: use the offline value function (tuned for given control and disturbance bounds) for safe control at run time.



3. HJ REACHABILITY: USING IT ONLINE

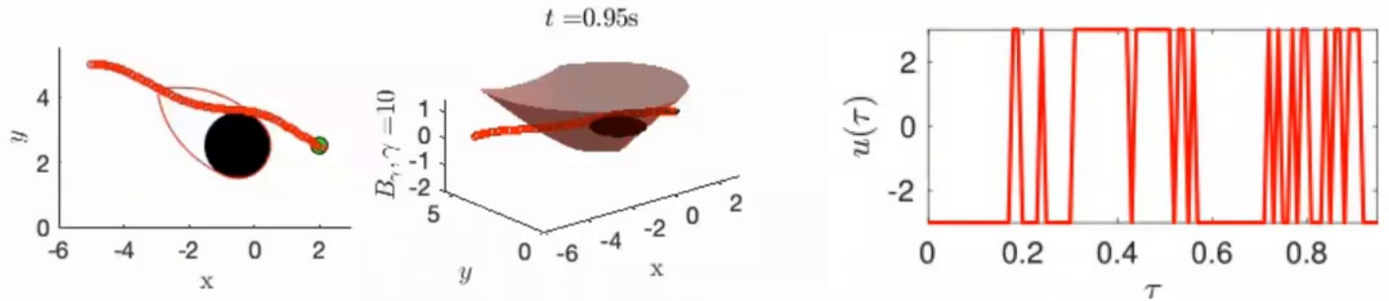
Online option 1: always use the safe policy

▶ 22:06

- Setup: 3D Dubins car (x, y, heading), a goal (green dot), a round obstacle. Plots: path, value function at the current heading, control u over time.
- Always following the value gradient only ever increases safety, so the car runs away from the obstacle and ignores the goal.
- Too conservative as a safety filter: it never lets you near the safe-set boundary, even when that would be fine.

Reachability Analysis for Safety Control:
Least-restrictive control law

- Switch to $\pi_V^*(\mathrm{x}(t),\ t)$ only when $\mathrm{x}(t)$ is close to the boundary of $\mathcal{S}(t)$.



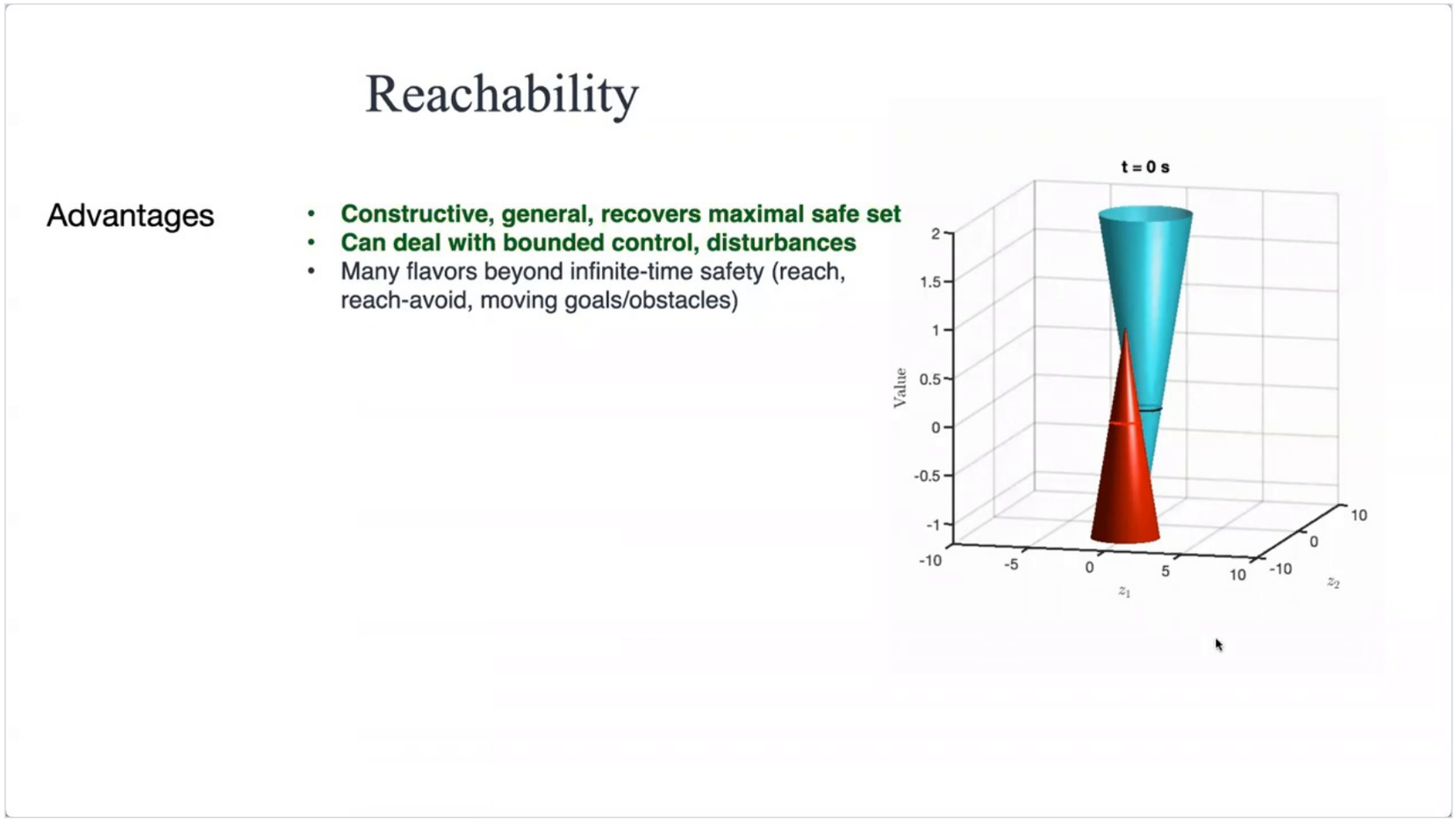
- Undesirable jerky behaviors
- Prone to errors in numerically computed $D_x V$.

3. HJ REACHABILITY: USING IT ONLINE

Online option 2: least-restrictive control

▶ 24:10

- Use the goal-seeking controller, and switch to the safety controller only when the state reaches the edge of the safe set S(t).
- Now the car reaches the goal, bouncing along the boundary.
- Downsides: the control chatters (red plot, rapid switching), which is jerky for motors, and it is sensitive to errors in the numerical gradient.
- In practice people blend the two controllers with a heuristic weight, rarely described in papers.



3. HJ REACHABILITY: USING IT ONLINE

Reachability: advantages

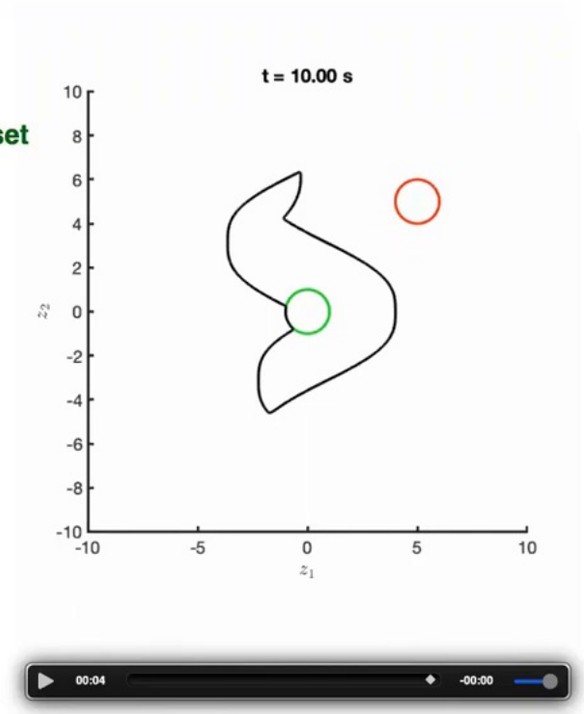
▶ 25:41

- **Constructive and general**: set up any cost and work backward with dynamic programming to get guarantees.
- Handles bounded control and disturbance directly, and recovers the **maximal** safe set.
- Many problem types: reach, reach-avoid, moving goals and obstacles. Example video: a goal plus a moving obstacle in one value function.

Reachability

Advantages

- **Constructive, general, recovers maximal safe set**
- **Can deal with bounded control, disturbances**
- Many flavors beyond infinite-time safety (reach, reach-avoid, moving goals/obstacles)



3. HJ REACHABILITY: USING IT ONLINE

Example: reach-avoid with a moving obstacle

▶ 26:53

- After 10 s: the reach-avoid set, the states guaranteed to reach the goal (green) while avoiding the moving obstacle (red) the whole time.
- Catch: you must know in advance exactly how obstacles and goals will move. Extra obstacles add no computation cost.

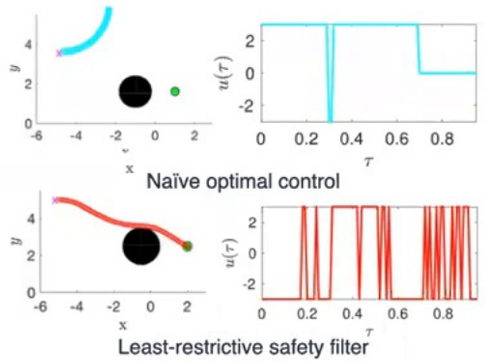
Reachability

Advantages

- **Constructive, general, recovers maximal safe set**
- Can deal with **bounded control, disturbances**
- Many flavors beyond infinite-time safety (reach, reach-avoid, moving goals/obstacles)

Disadvantages

- Online deployment of the value function is not straightforward **(either too conservative or jerky)**

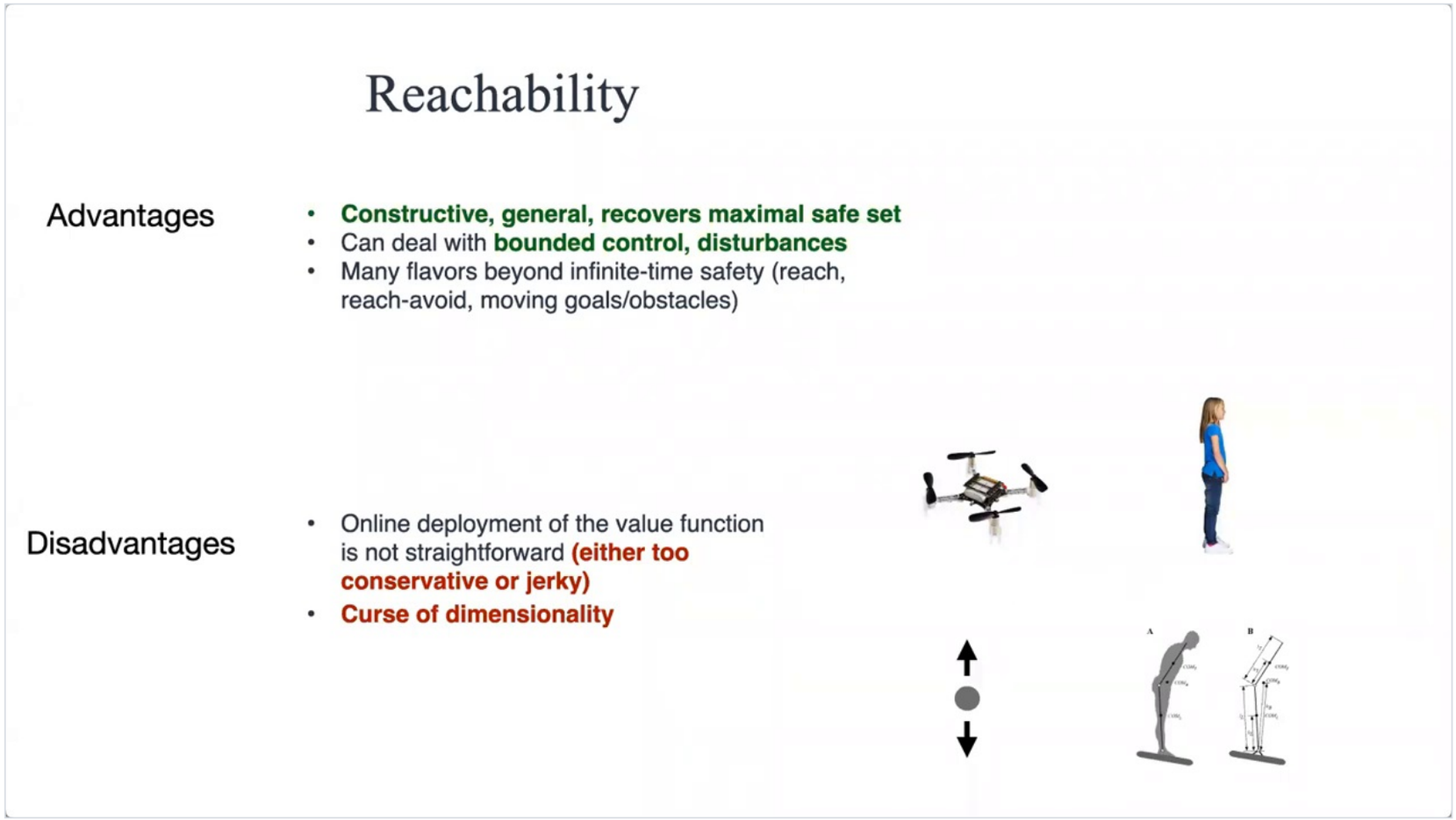


3. HJ REACHABILITY: USING IT ONLINE

Reachability disadvantage 1: online use

▶ 27:17

- Using the value function online is awkward: either too conservative (naive optimal control, top plots) or jerky (least-restrictive filter, bottom plots).
- People usually blend the two by hand.

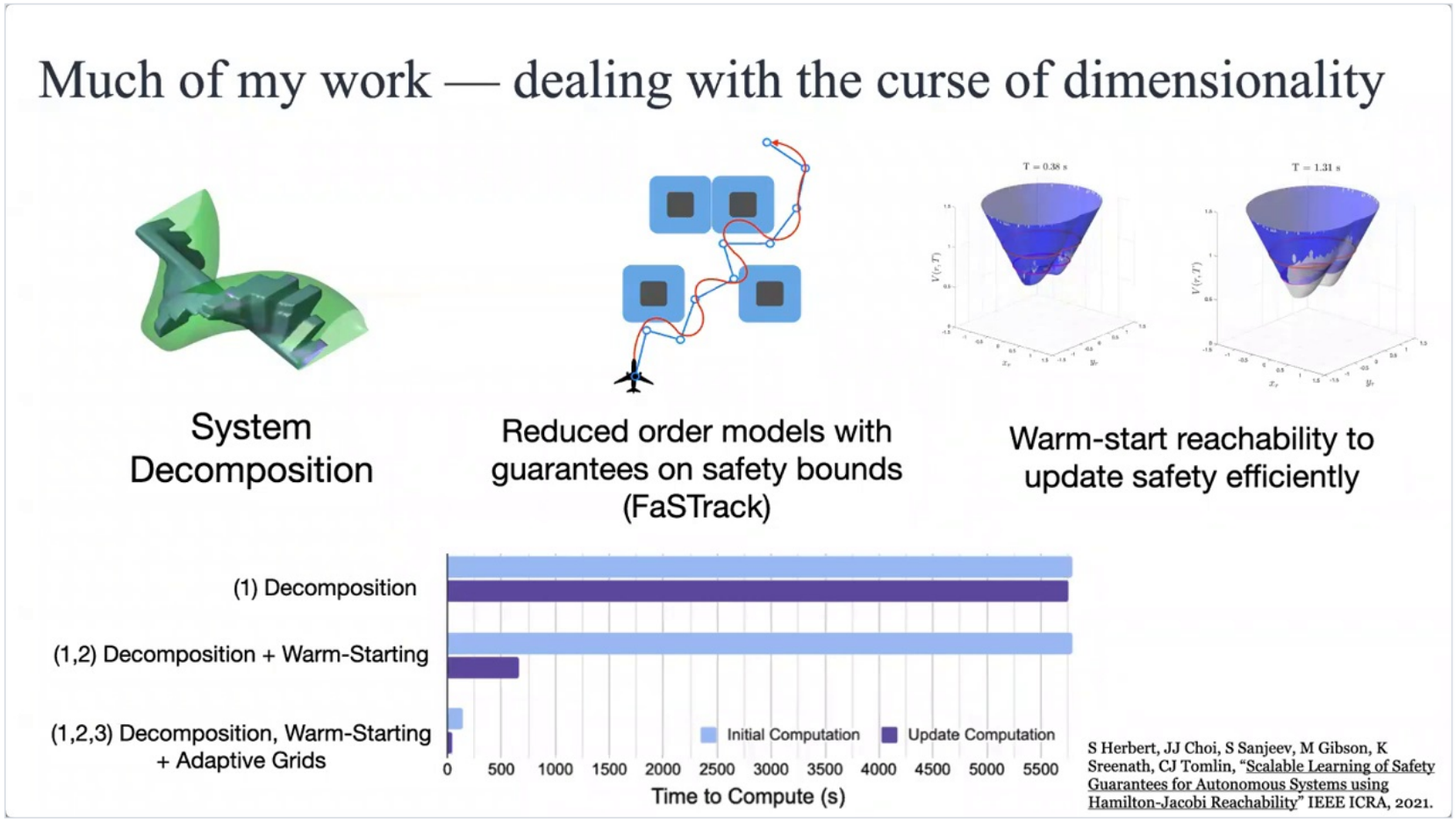


3. HJ REACHABILITY: USING IT ONLINE

Reachability disadvantage 2: curse of dimensionality

▶ 27:34

- Dynamic programming updates every point of a uniform grid at every time step, so each extra state dimension multiplies the work. Example: 100 points per axis gives 10^6 points in 3D but 10^{12} in 6D.
- So people analyze simplified models: a 2D point mass instead of a quadcopter, a double inverted pendulum instead of a human body. Results may not hold for the real system.
- Today's best toolboxes handle about 5–6 state dimensions directly.



3. HJ REACHABILITY: USING IT ONLINE

Her work on the curse of dimensionality

▶ 28:55

- **System decomposition**: split the system into subsystems (exactly or approximately), analyze each, recombine.
- **Reduced-order models with guarantees** (FaSTrack): plan with a simple model and bound how far the real system can drift from it.
- **Warm-start reachability**: when bounds or obstacles change, update the old value function instead of starting over.
- ICRA 2021, 10D quadcopter: decomposition alone ≈ 5,700 s per computation; + warm start: updates ≈ 650 s; + adaptive grids: ≈ 130 s first, ≈ 40 s per update. Better, but still "a thorn in our side".



4. CONTROL BARRIER FUNCTIONS

Part 2 starts: control barrier functions

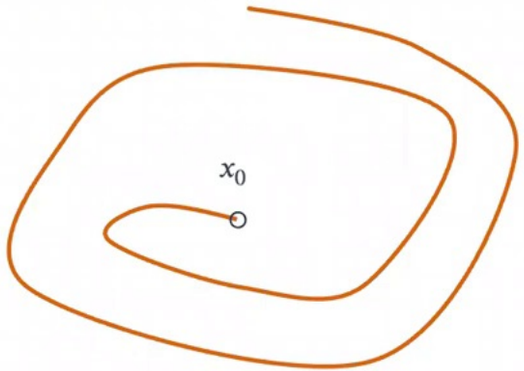
▶ 31:25

- Section divider before moving to CBFs. Most of this time is a Q&A about online reachability control.

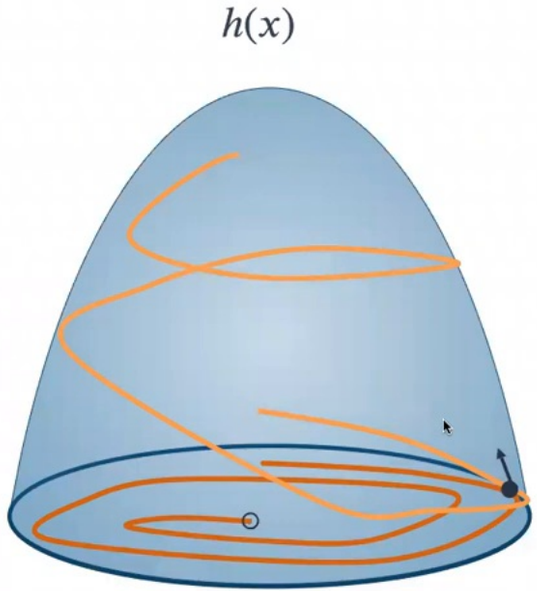
Q&A: *Sahil asked why following the safety gradient is conservative, and whether goal and avoidance can be combined without a hand-set threshold. Answer: the value function only encodes the obstacle, so its gradient always points away from it. In practice people blend heuristically; Karen Leung and Mo Chen (Pavone's lab) took the best-performing action that keeps safety from decreasing. The CBF community already does this kind of blending.*

Nagumo’s Invariance Principle

$\dot{x} = f(x)$



$\mathscr{C} = \{x \mid h(x) \geq 0\}$
is invariant!



$\dot{h}(x) \geq 0$
$\forall\, x \in \partial\mathscr{C}$

4. CONTROL BARRIER FUNCTIONS

Nagumo’s invariance principle

▶ 34:17

- CBFs build on Nagumo’s theorem: for dx/dt = f(x), the set C = {x | h(x) ≥ 0} is invariant (trajectories never leave) if dh/dt ≥ 0 on the boundary of C.
- Inside C, h may go down. It only has to stop decreasing at the boundary.

Control Barrier Functions

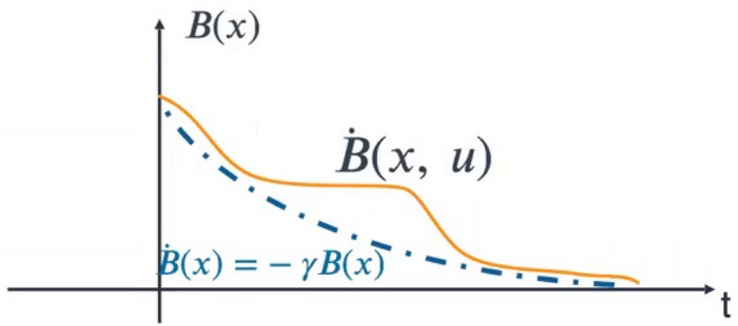
Let $B(x): \mathbb{R}^n \to \mathbb{R}$ a continuously differentiable function whose zero-superlevel set is $\mathscr{C}$

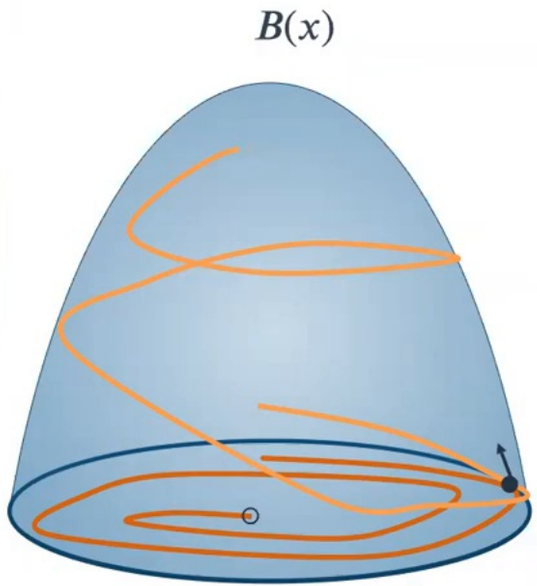
$$(\mathscr{C} = \{x \mid B(x) \geq 0\}) \text{ and } \nabla B(x) \neq 0 \text{ for all } x \in \partial C.$$

If there exists a positive coefficient γ such that for all $x \in \mathscr{C}$,

$$\sup_{u \in U} \dot{B}(x, u) + \gamma B(x) \geq 0$$

then $B(x)$ is a **Control Barrier Function** and any Lipschitz continuous control law that satisfies the above constraint will render the set $\mathscr{C}$ safe (i.e. **control invariant**)*.





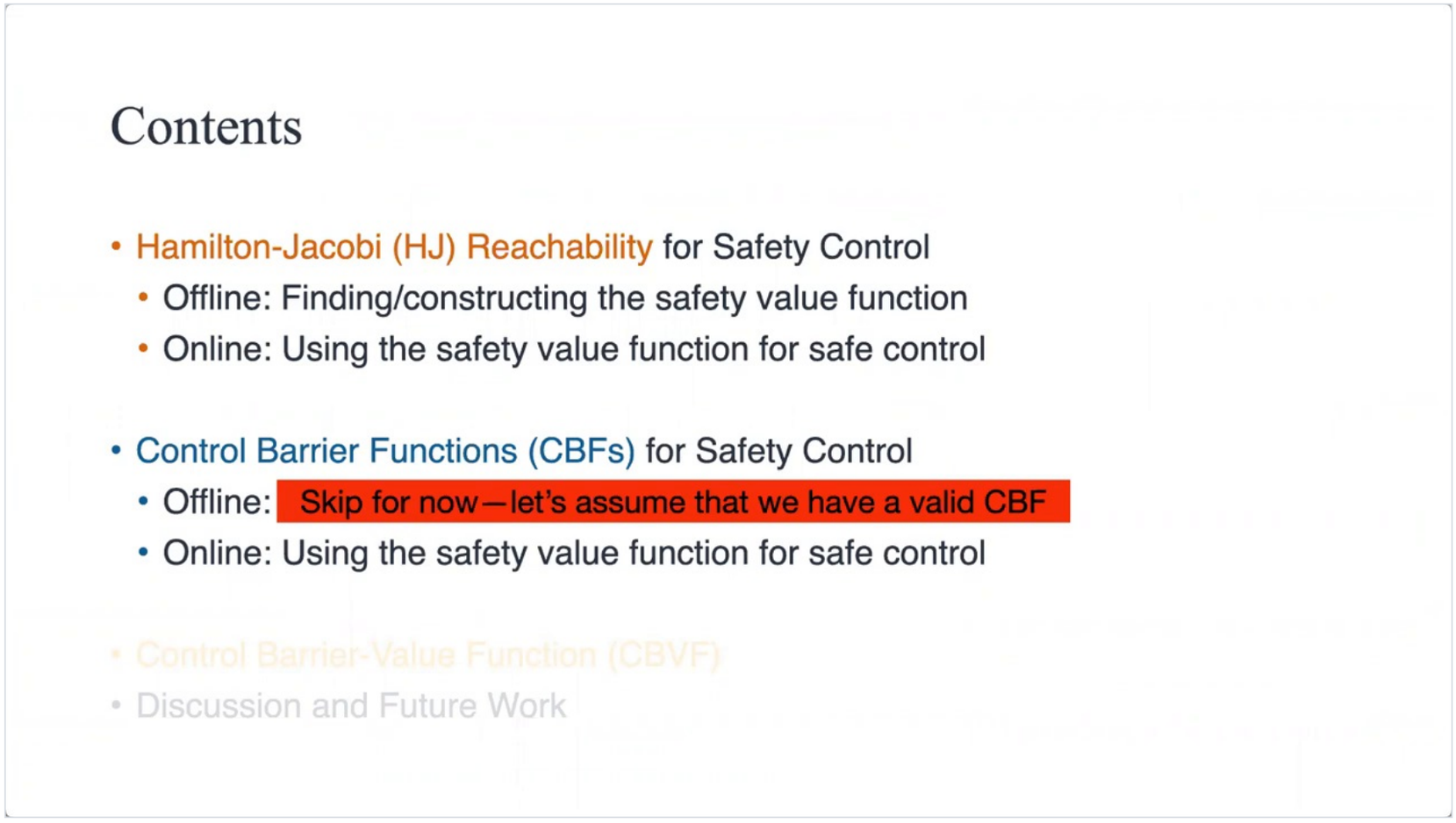
$$\dot{B}(x, u) + \gamma B(x) \geq 0$$

4. CONTROL BARRIER FUNCTIONS

Control barrier function: definition

▶ 34:50

- B: $\mathbb{R}^n \to \mathbb{R}$ is continuously differentiable, its safe set is C = {x | B(x) ≥ 0}, and ∇B ≠ 0 on the boundary.
- B is a CBF if some γ > 0 gives $\sup_{u \in U}$ dB/dt(x,u) + γB(x) ≥ 0 for all x in C. Then any Lipschitz control meeting this keeps C safe.
- Key difference from reachability: safety may drop, but no faster than −γB. Example, γ = 2: at B = 3, dB/dt ≥ −6; at B = 0.1, dB/dt ≥ −0.2; at B = 0, dB/dt ≥ 0.
- This builds in the “loose far away, strict near the edge” blending from the Q&A. Another difference: B must be differentiable everywhere.

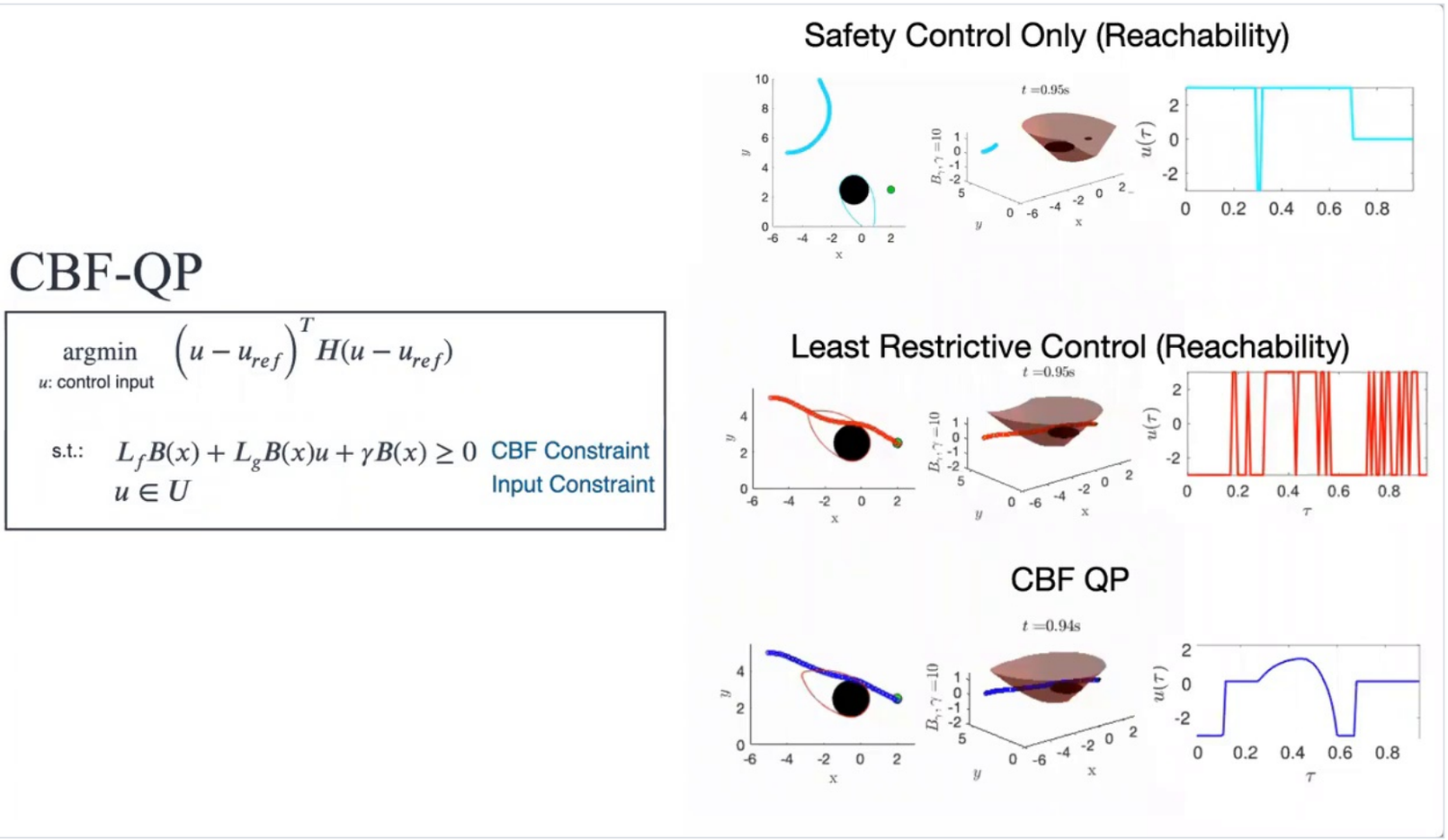


4. CONTROL BARRIER FUNCTIONS

CBFs offline: skipped for now

▶ 37:55 · revisited at 40:29

- How to find a valid CBF is postponed ("assume we have one"). The talk covers online use first and returns to this at 40:29.

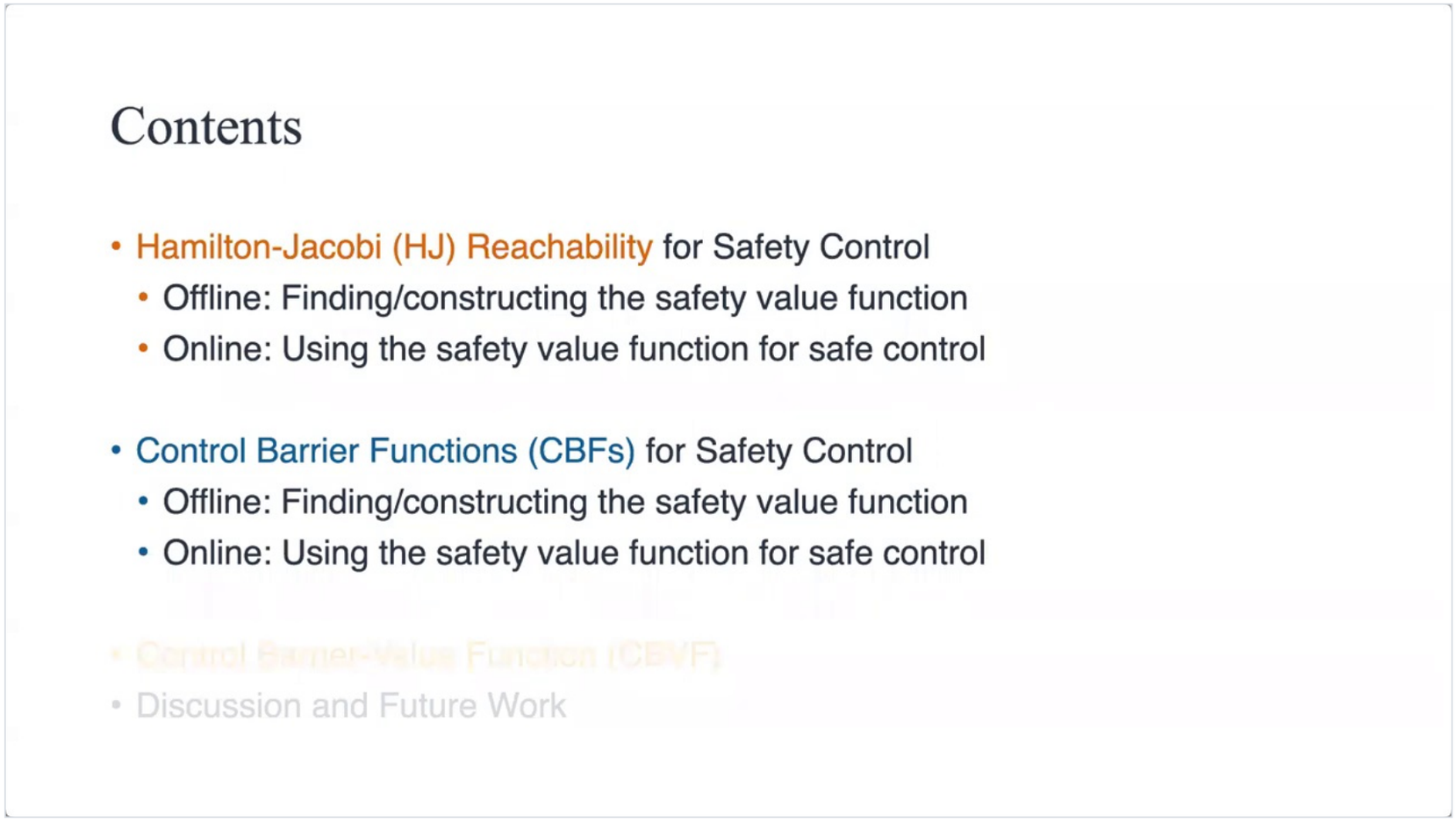


4. CONTROL BARRIER FUNCTIONS

CBF-QP: online safety filter

▶ 38:13

- A QP picks the control closest to the reference u_{ref}, subject to the CBF constraint $L_fB + L_gB \cdot u + \gamma B \geq 0$ and $u \in U$. (L_fB, L_gB: how B changes along the drift and input directions of $dx/dt = f(x) + g(x)u$.) Fast to solve for control-affine systems.
- Far from the edge it just follows the performance controller; near the edge the constraint tightens and steers toward safety.
- Dubins car comparison: safety-only runs away; least-restrictive reaches the goal with chattering; CBF-QP takes a similar path with smooth control. Reachability should borrow this.

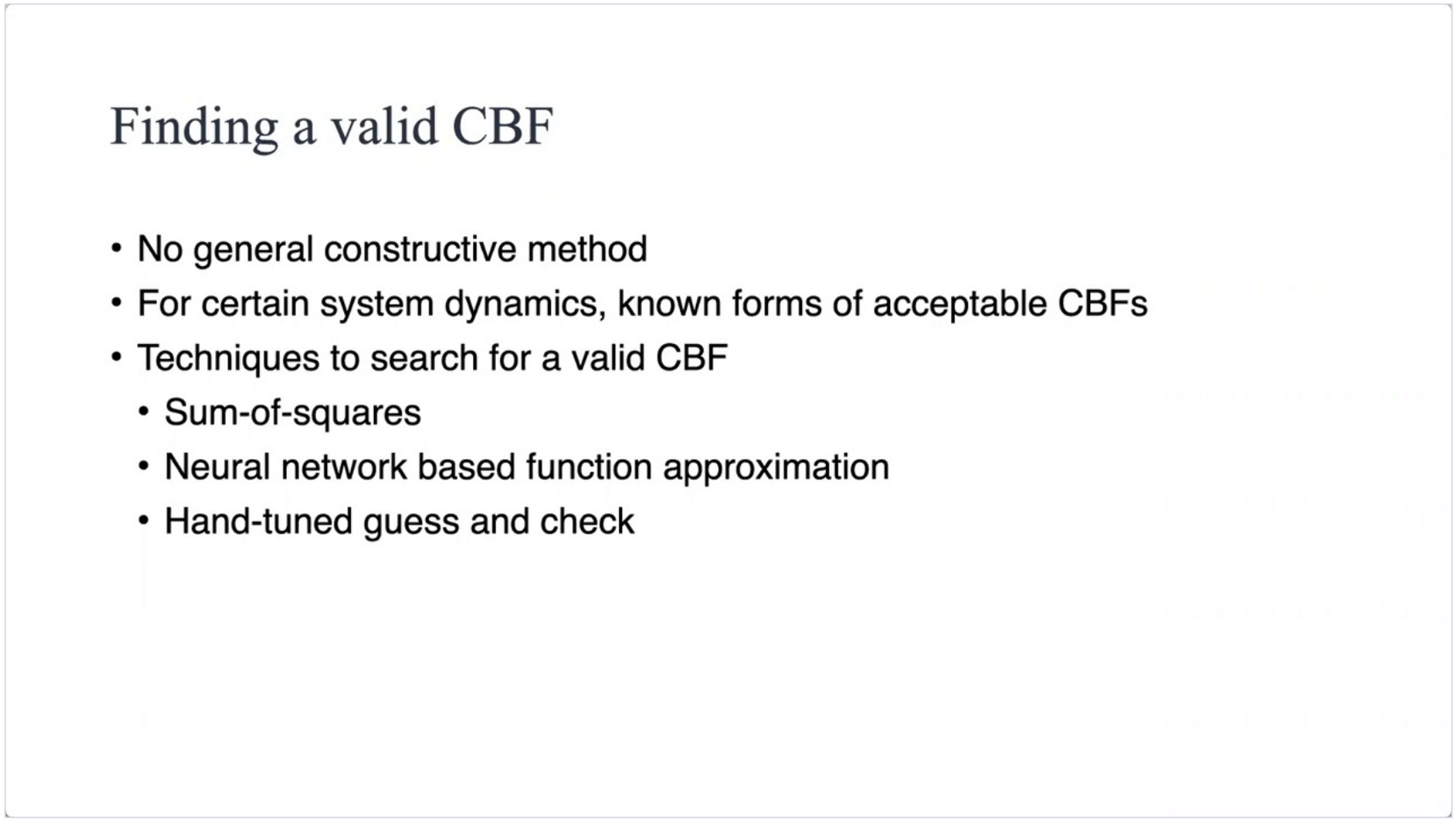


4. CONTROL BARRIER FUNCTIONS

CBFs, offline: finding one

▶ 40:37

- Back to the skipped question: how to get a CBF that meets the safety constraint, the control bounds, and so on. This is where CBFs struggle.



4. CONTROL BARRIER FUNCTIONS

Finding a valid CBF

▶ 40:51

- No general constructive method exists.
- For some dynamics, known CBF forms exist: guess one, then check the conditions.
- Search methods: sum-of-squares optimization; neural networks fit to sampled points (mostly near the boundary), retrained on counterexamples; hand-tuned guess and check.
- Sometimes a smooth, conservative function is simply used without a formal proof.

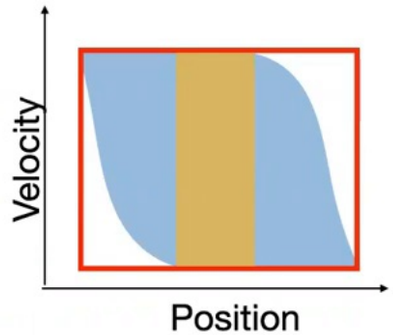
CBFs

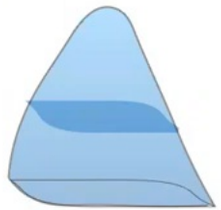
Advantages

- **CBF-QP nicely trades off** between the performance and safety controller
- Not trying to find the maximal safe set—often use simpler functions that are more conservative but also **more intuitive, easy to use** (e.g. defined only in position space)

Disadvantages

- **No general constructive method**
- Typically recover a **subset of the maximal safe set**
  - Guess-and-check looks only for valid CBFs, not the "best" CBF
  - The community requires that the function is differentiable everywhere (reachability community is fine with points of non-differentiability)
- Typically **hard to encode control bounds, disturbance bounds, etc.**





Reachability value function defined in 2D



CBF defined in 1D

4. CONTROL BARRIER FUNCTIONS

CBFs: pros and cons

▶ 42:06

- **Pros**: the CBF-QP trades off performance and safety nicely. Not chasing the largest safe set allows simpler functions, e.g., a smooth 1D function of position only (orange) instead of a 2D position-velocity set with kinks.
- **Cons**: no general construction, and with control and disturbance bounds the QP may become infeasible online.
- Usually only part of the maximal safe set, for two reasons: guess-and-check finds *a* valid CBF, not the best; and CBFs must be smooth, while reachability allows kinks (riding straight at a tree: turn left or right?) using viscosity solutions.
- Control and disturbance bounds are hard to encode.

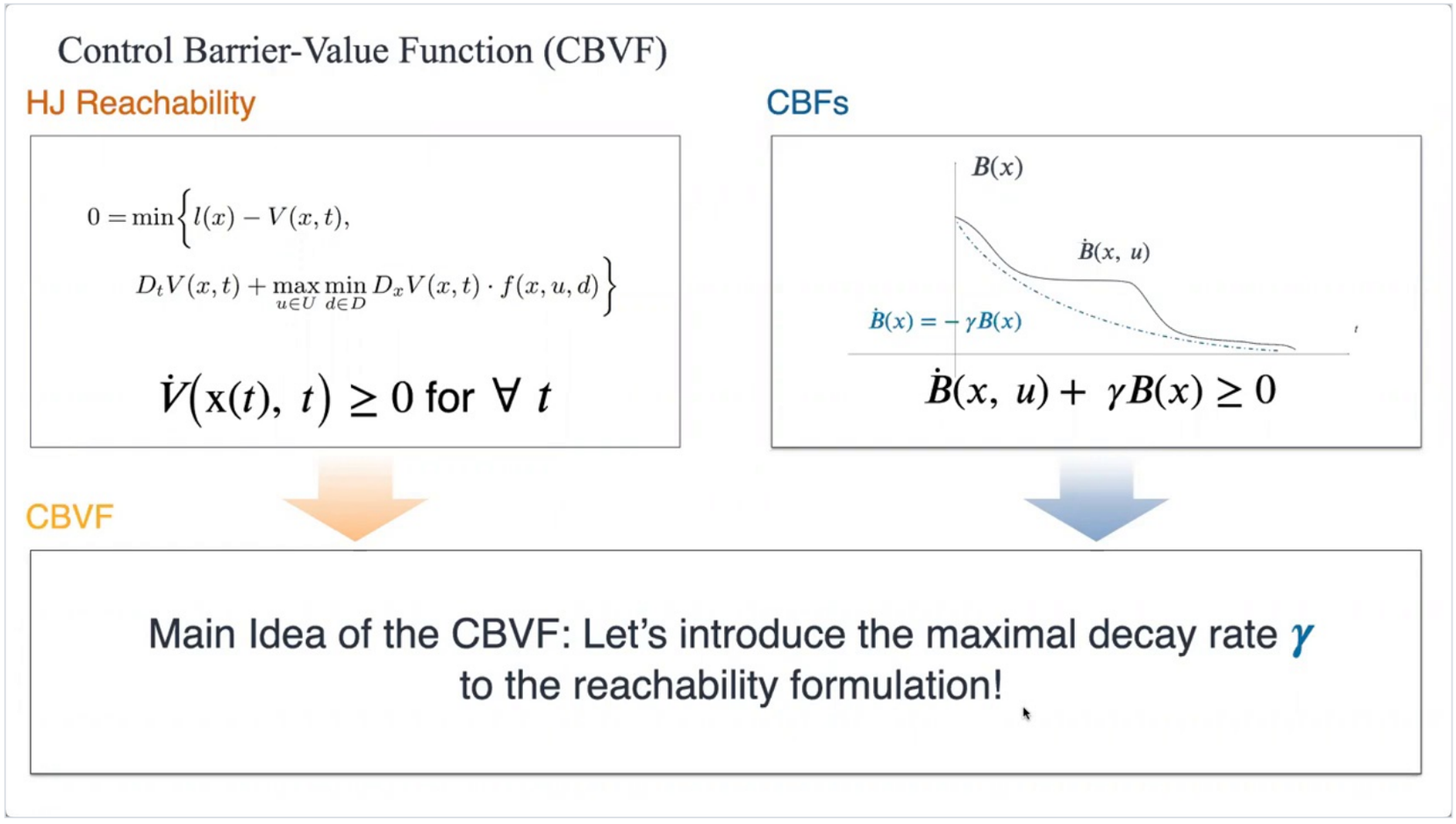
	Reachability	CBFs
Advantages	• **Constructive, general, recovers maximal safe set** • Can deal with **bounded control, disturbances** • Many flavors beyond infinite-time safety (reach, reach-avoid, moving goals/ obstacles)	• **CBF-QP nicely trades off** between the performance and safety controller • Not trying to find the maximal safe set—often use simpler functions that are more conservative but also **more intuitive, easy to use** (e.g. defined only in position space)
Disadvantages	• Online deployment of the value function is not straightforward **(either too conservative or jerky)** • **Curse of dimensionality**	• **No general constructive method** • Typically recover a **subset of the maximal safe set** • Guess-and-check looks only for valid CBFs, not the "best" CBF • The community requires that the function is differentiable everywhere (reachability community is fine with points of non-differentiability) • Typically **hard to encode control bounds, disturbance bounds, etc.**

5. MERGING THEM: THE CBVF

Reachability vs. CBFs

▶ 46:47 · revisited at 48:52

- Side-by-side summary of the two previous pros/cons slides.
- Goal of the CBVF: keep the advantages of both, by changing only the safety constraint that reachability encodes.

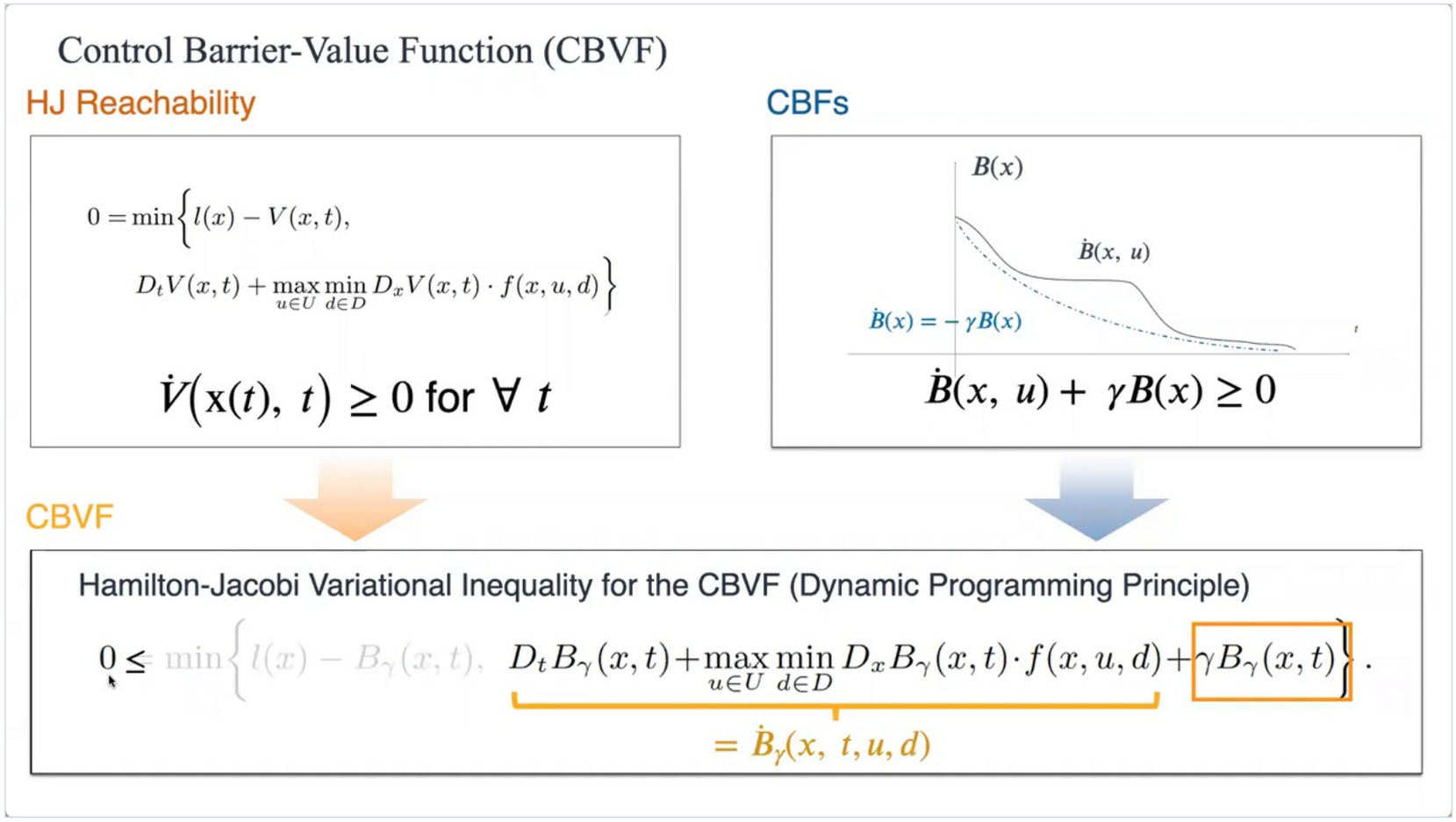


5. MERGING THEM: THE CBVF

CBVF: main idea

▶ 46:58

- The key difference is the constraint: reachability asks dV/dt ≥ 0; CBFs ask dB/dt + γB ≥ 0.
- Idea: put the CBF's maximal decay rate γ into the reachability formulation.



5. MERGING THEM: THE CBVF

HJ-VI for the CBVF

▶ 47:44

- Take the reachability HJ-VI and add $+\gamma B_\gamma$: $D_t B_\gamma + \max_u \min_d D_x B_\gamma \cdot f(x,u,d) + \gamma B_\gamma$, alongside the $l(x) - B_\gamma$ term.
- The first part is dB_γ/dt, so the equation enforces $dB_\gamma/dt + \gamma B_\gamma \geq 0$.

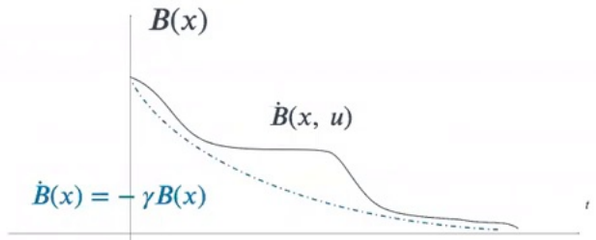
Control Barrier-Value Function (CBVF)

HJ Reachability

$$0 = \min\Big\{ l(x) - V(x,t), \quad D_t V(x,t) + \max_{u \in U} \min_{d \in D} D_x V(x,t) \cdot f(x,u,d) \Big\}$$

$$\dot{V}(\mathrm{x}(t),\ t) \geq 0 \text{ for } \forall\ t$$

CBFs



$$\dot{B}(x,\ u) + \gamma B(x) \geq 0$$

CBVF

$$\dot{B}_\gamma(\mathrm{x}(t),\ t, u, d) + \gamma B_\gamma(\mathrm{x}(t), t) \geq 0 \text{ for } \forall d \in D,\ \forall\ t$$

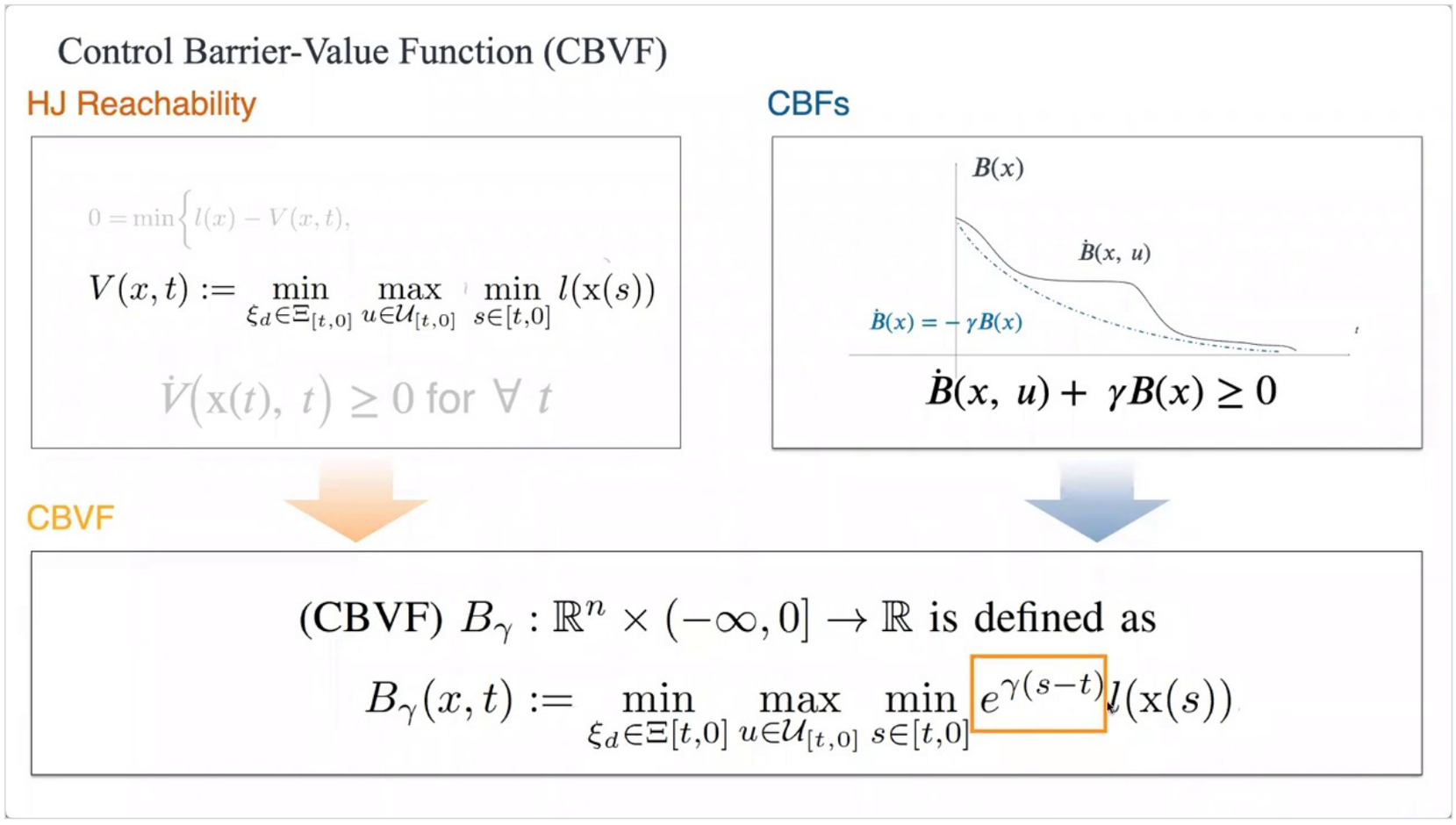
under the optimal control policy for B_γ, $u = \pi_B^*(x,\ t)$.

5. MERGING THEM: THE CBVF

What the CBVF guarantees

▶ 48:09

- $dB_\gamma/dt + \gamma B_\gamma \geq 0$ for every disturbance d in D and all t, under the optimal policy $u = \pi^*_B(x,t)$.
- Same dynamic programming as before; only the encoded constraint changes from “≥ 0” to “≥ −γB”.

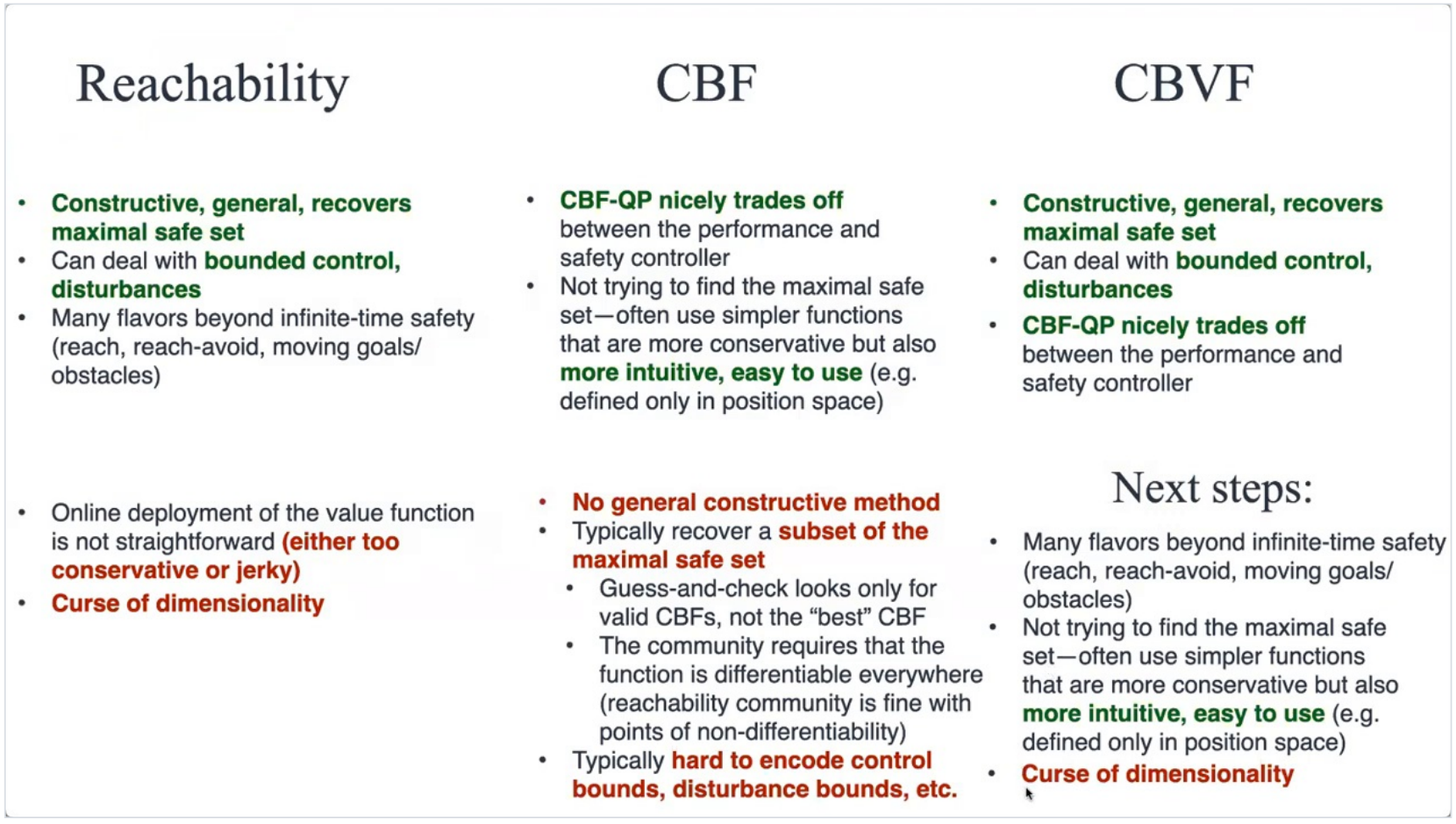


5. MERGING THEM: THE CBVF

CBVF definition

▶ 48:39

- Reachability value: V(x,t) = min over disturbance, max over control, of the smallest l(x(s)) along the trajectory.
- CBVF: B_γ(x,t) is the same, except each l(x(s)) is weighted by $e^{\gamma(s-t)}$.
- With γ = 0 the weight is 1 and you get the reachability value back.



5. MERGING THEM: THE CBVF

Reachability vs. CBF vs. CBVF, and next steps

▶ 48:57 · revisited at 50:27, 52:33, 53:43

- CBVF column: constructive and general, recovers the maximal safe set, handles bounded control and disturbance (from reachability), and works in a QP filter (from CBFs).
- Called CBVF, not CBF, because it can still have non-differentiable points. In practice it acts like a CBF.
- Next steps (added at 50:27 and 52:33): more flavors beyond infinite-time safety; borrow the CBF habit of simpler, conservative functions; and keep attacking the curse of dimensionality.

CBVF-QP: Online optimal control synthesis for Control-Affine systems.

$$\dot{x} = p(x) + q(x)u + r(x)d$$

Robust CBVF-QP:

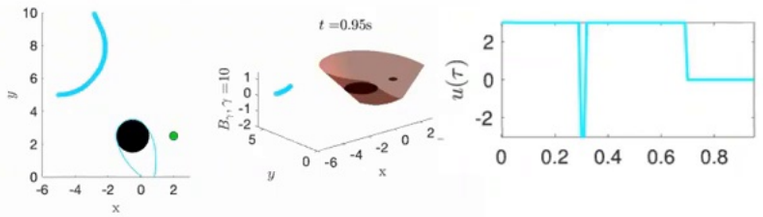
$$\pi_{QP}(x,t) = \arg\min_{u \in U} \quad (u - u_{ref})^T (u - u_{ref}) \tag{20a}$$

$$\text{s.t.} \quad a(x,t) + D_x B_\gamma(x,t) \cdot q(x)u + \gamma B_\gamma(x,t) \geq 0, \tag{20b}$$

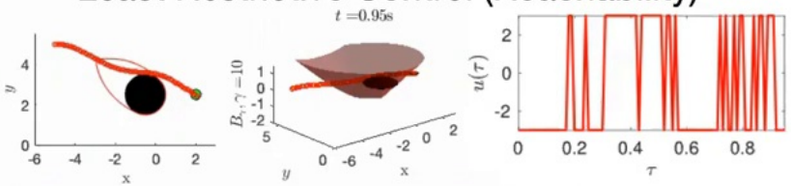
$$\text{where } a(x,t) = D_t B_\gamma(x,t) + D_x B_\gamma(x,t) \cdot p(x) + \min_{d \in D} D_x B_\gamma(x,t) \cdot r(x)d. \tag{20c}$$

- Robust CBVF-QP gives the optimal control signal no matter what u_{ref} is used.
- Can satisfy control, disturbance constraints that are built-in
- **This can be directly used as a safety filter.** We don't have to do least-restrictive switching control

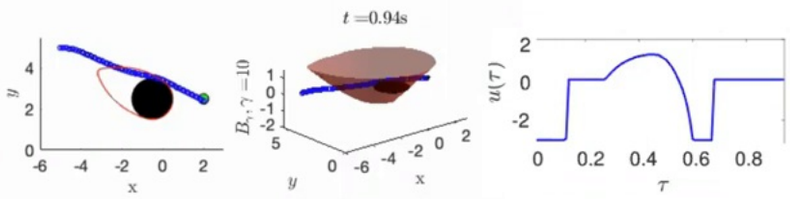
Safety Control Only (Reachability)



Least Restrictive Control (Reachability)



CBVF QP



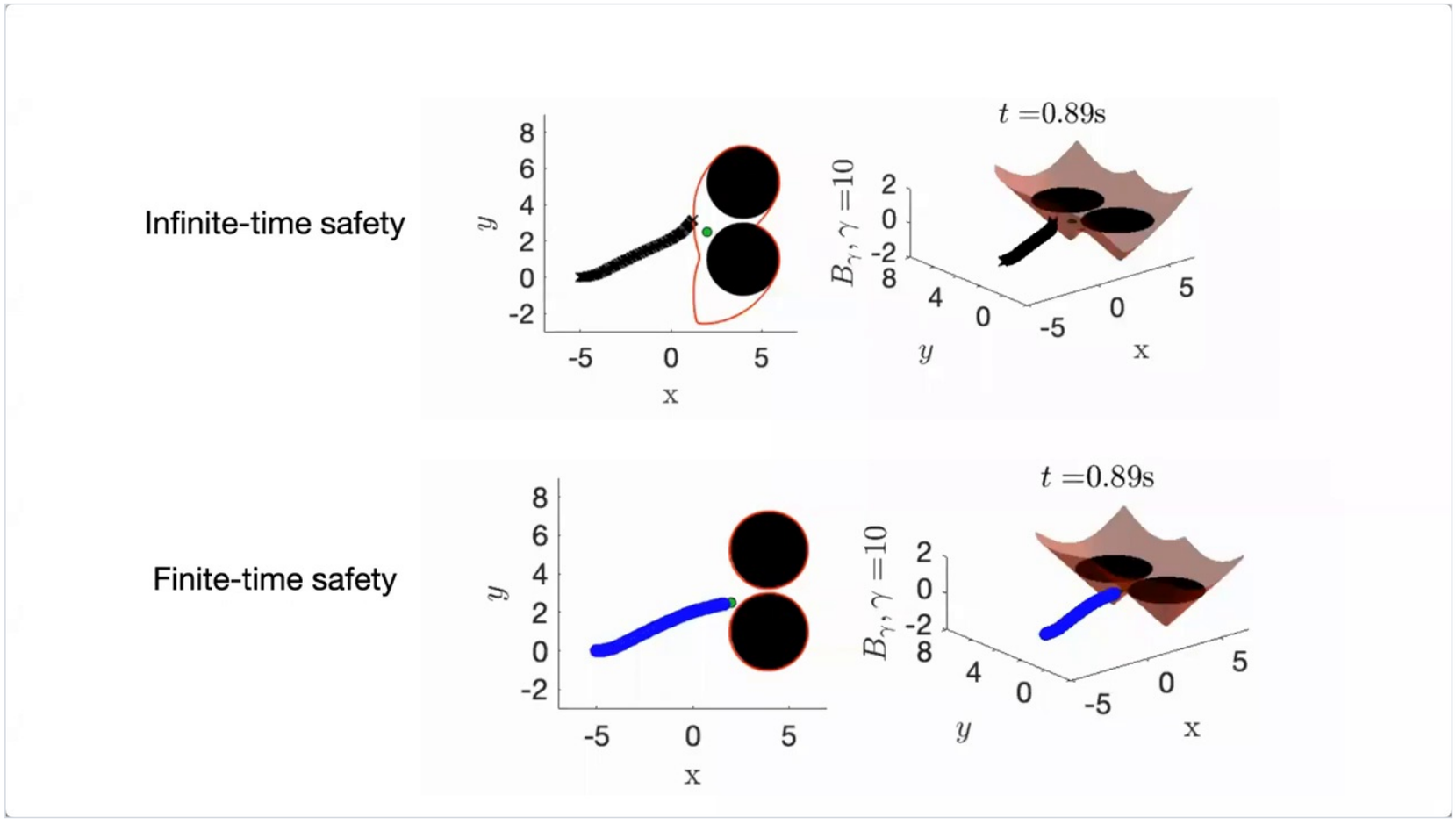
5. MERGING THEM: THE CBVF

Robust CBVF-QP

▶ 49:33 · revisited at 1:02:31

- For dx/dt = p(x) + q(x)u + r(x)d: pick u closest to u_{ref}, subject to a(x,t) + $D_x B_\gamma \cdot q(x)u + \gamma B_\gamma \geq 0$, where a(x,t) assumes the worst-case disturbance.
- Gives the optimal safe control for any u_{ref}, with control and disturbance limits built in. Works directly as a safety filter, no switching needed.
- The bottom video uses a real CBVF: smooth control, reaches the goal.

Q&A: *(62:31) Why did CBVF-QP reach the goal slightly faster (0.94 s) than least-restrictive (0.95 s)? Specific to this setup: the goal is behind the obstacle, so turning early is shorter. Usually least-restrictive is more direct, but smooth control is worth a slower path. Does the QP plan ahead like MPC? No; CBF constraints can be added to MPC. If the goal is known in advance, reachability can build one reach-avoid value function. Chat question: give a broken drone a finite cost instead of a hard constraint? An interesting idea she hasn't explored.*



6. FUTURE WORK AND QUESTIONS

Infinite- vs. finite-time safety

▶ 50:40

- Two obstacles with the goal between them. Infinite-time safety is too strict: reaching the goal would eventually force a crash, so it never lets the car get there.
- Finite-time safety: if you only need to stay safe until reaching the goal (e.g., a hybrid system then switches mode), the safe set shrinks over time and the car reaches the goal.
- Useful for walking robots (Sreenath's work), which switch modes at each step.

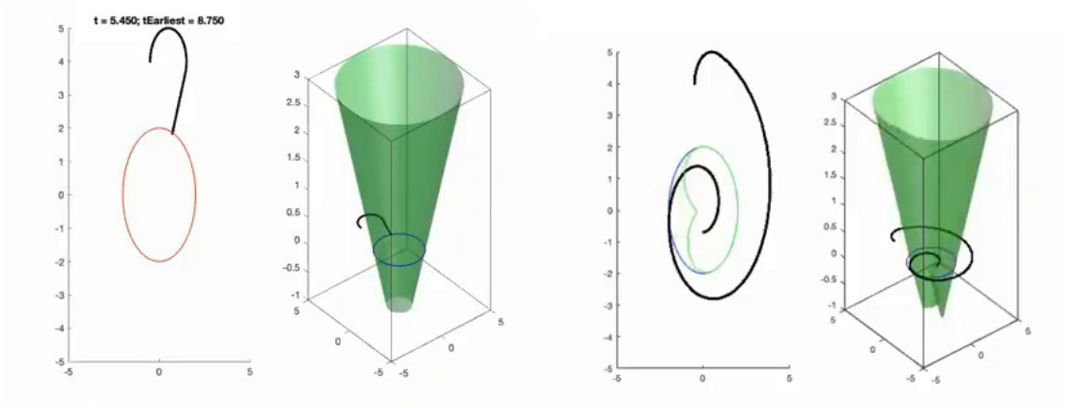




60

Reachability & Control Lyapunov Functions

Merging the theory and tools from HJ reachability and control Lyapunov functions (CLFs)



Reachable Tube
(minimum time to goal)

Modified Infinite-Horizon
Avoid Tube (stabilize to goal)

6. FUTURE WORK AND QUESTIONS

Reachability and control Lyapunov functions

▶ 51:56

- The “reach” counterpart of a CBF is a control Lyapunov function (CLF), which drives the system to a goal and keeps it there.
- Standard reachable tube: reach the goal in minimum time, but not necessarily stay. Modified infinite-horizon avoid tube: enter the goal and stay there stably, like a CLF.

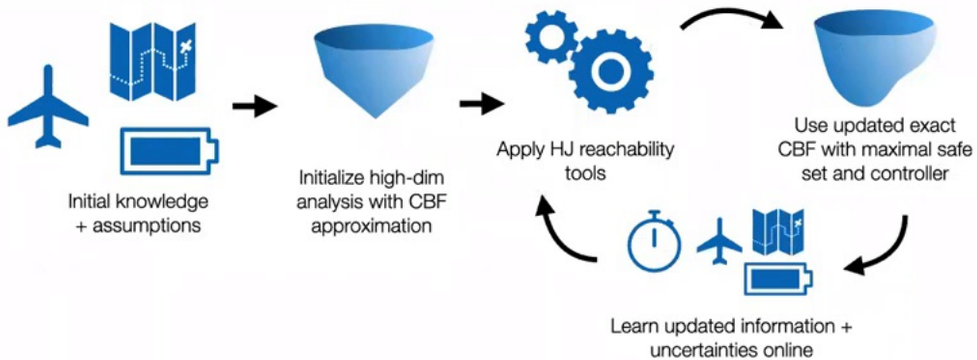


64

Safe Learning with CBFs

Take advantage of tools for approximating high-dimensional safety functions from the CBF community

Take advantage of tools for dealing with curse of dimensionality from reachability community (decomposition, warm-starting, reduced-order modeling, etc.)



6. FUTURE WORK AND QUESTIONS

Safe learning with CBFs

▶ 52:54

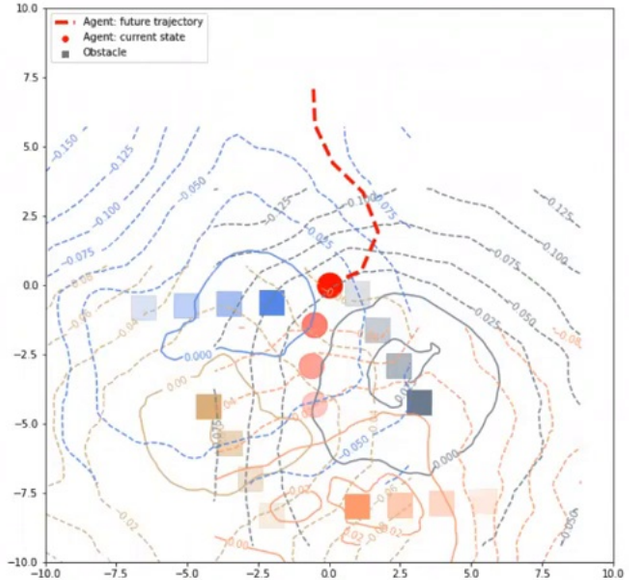
- Combine CBF tools for approximating high-dimensional safety functions with reachability tools for the curse of dimensionality (decomposition, warm starting, reduced-order models).
- Loop: initial knowledge → CBF-style high-dimensional approximation → HJ reachability updates to make it exact → learn online → update and repeat.





65

Generalizing CBFs for multi-agent interactions
Reasoning about uncertainty online



6. FUTURE WORK AND QUESTIONS

Multi-agent CBFs

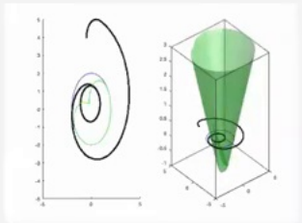
▶ 53:29

- Collaboration with Prof. Gao (UCSD computer science) on generalizing CBFs to multi-agent interactions and reasoning about uncertainty online.
- Skipped for time.

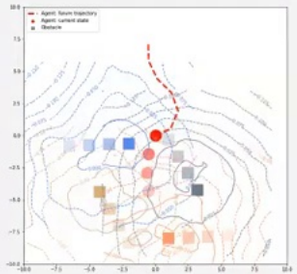
Thank you!











Questions, ideas, complaints, etc.:
sherbert@ucsd.edu

CBVF

- **Constructive, general, recovers maximal safe set**
- Can deal with **bounded control, disturbances**
- **CBF-QP nicely trades off** between the performance and safety controller

Next steps:

- Many flavors beyond infinite-time safety (reach, reach-avoid, moving goals/ obstacles)
- Not trying to find the maximal safe set—often use simpler functions that are more conservative but also **more intuitive, easy to use** (e.g. defined only in position space)
- **Curse of dimensionality**

6. FUTURE WORK AND QUESTIONS

Thank you and questions

▶ 53:46 · revisited at 1:07:27

- Thanks to the CBVF collaborators and the students on the next-step projects. Contact: sherbert@ucsd.edu.

Q&A: *What if the system ends up outside the safe set? A CBF defined there acts like a CLF and pulls the state back (Aaron Ames); reachability hasn't studied this recovery yet. Discussion: safety limits are set conservatively, so "unsafe" is not always fatal; how do we manage unsafe states systematically, since you can't reboot a plane? Herbert agreed the field must go beyond hard guarantees: bees bump off grass and recover, and surgical robots must touch tissue. A later question goes back to the CBVF-QP slide (page 40).*